



Basic Syntax

What is Syntax?

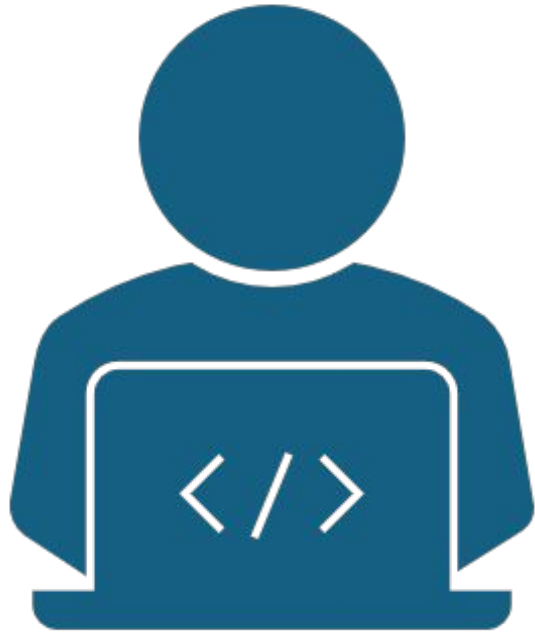
- Is the set of rules that dictate how to write computer commands in a specific programming language.
- It includes the order of instructions, punctuation, keywords, and symbols.

File Edit View

```
<html>
<head>
  <title>My First HTML Page</title>
</head>
<body>
  <h1>Hello World</h1>
</body>
</html>
```

Why is Syntax Important?

- **Consistency:** Syntax ensures that code is written in a consistent way, making it easier for a compiler or interpreter to understand.
- **Readability:** Following syntax rules helps developers write readable code.
- **Avoid errors:** Following syntax rules helps developers avoid errors that can cause a program to crash or run incorrectly.



Basic Syntax:

1. Variables

2. Data Types

1. list

2. tuple

3. dict

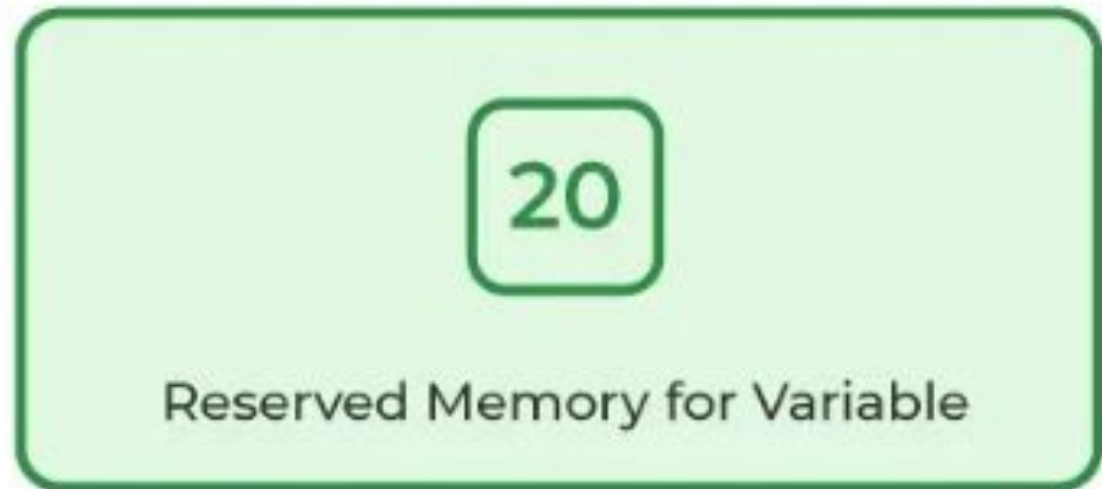
4. set

1. functions

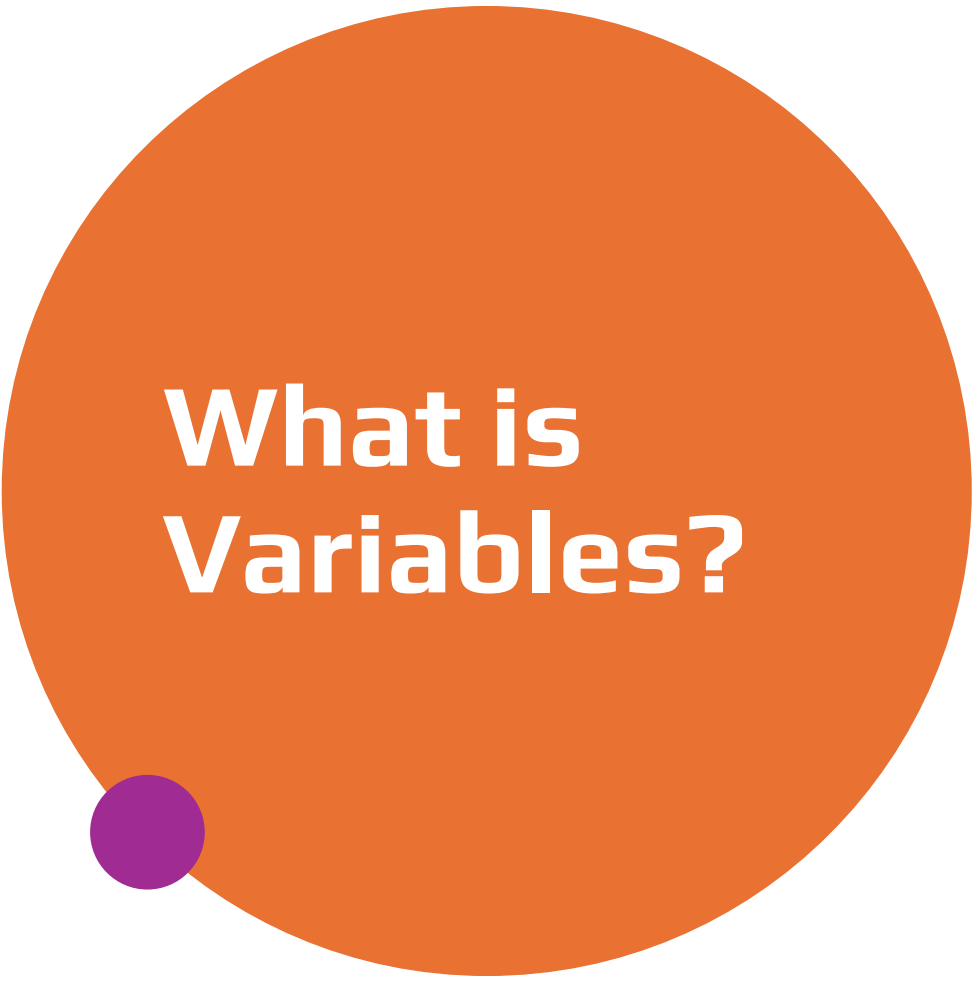
Variables

Int age = 20;

Data Variable_name Value
Type

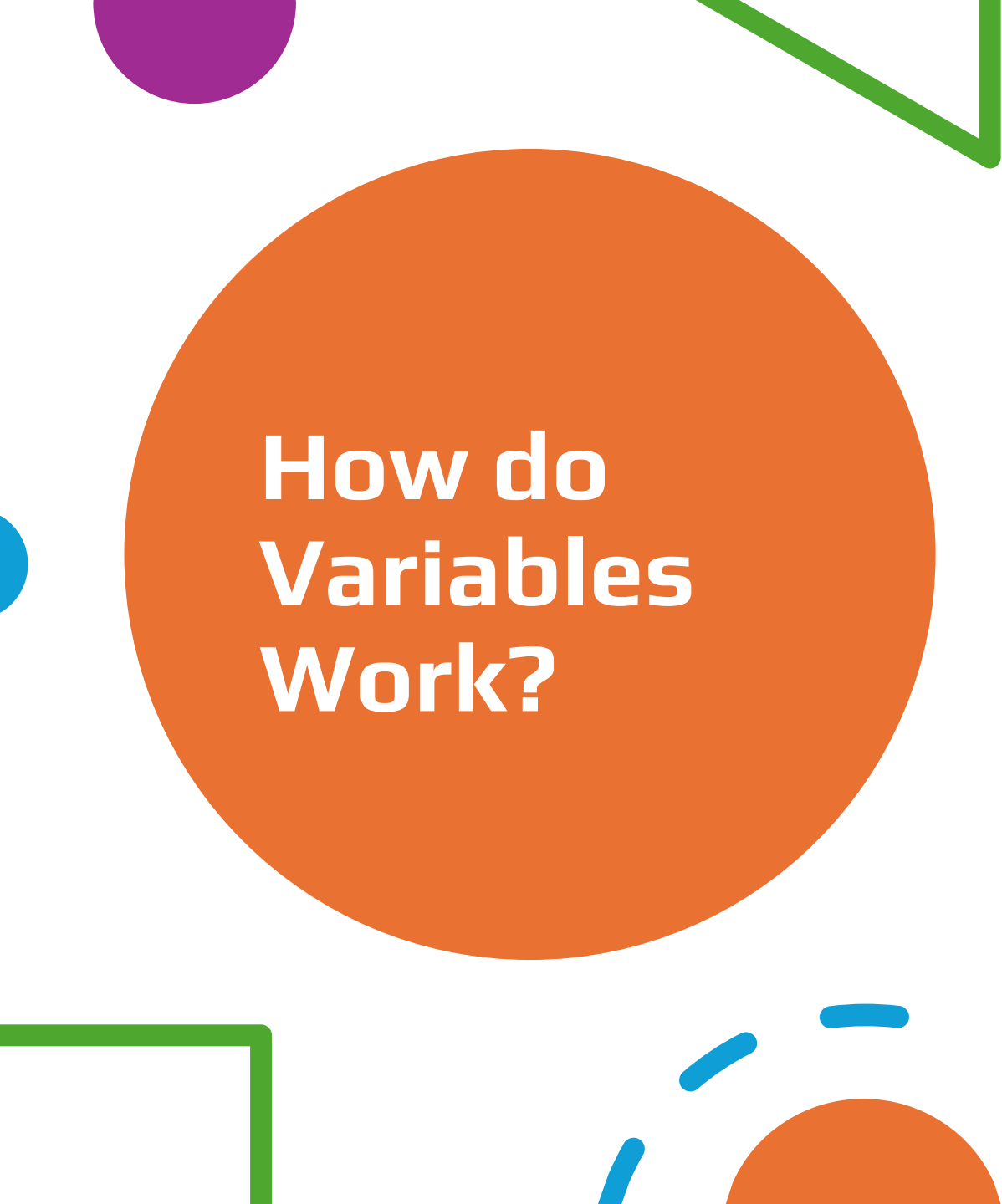


RAM



What is Variables?

- In programming, a variable is a named storage location that can hold a value that changes.
- Variables are essential for modern computing and allow developers to write code more efficiently.
- A variable can refer to anything from numbers and strings to objects, collections, and pointers.



How do Variables Work?

- Variables can store numbers, strings, objects, collections, and pointers.
- Variables can be associated with a memory address.
- Variables can be defined as a specific data type, such as an integer or a string.
- Variables can be used to store values that can change based on conditions or user input.
- Variables can be used to store values that are reused across multiple functions.

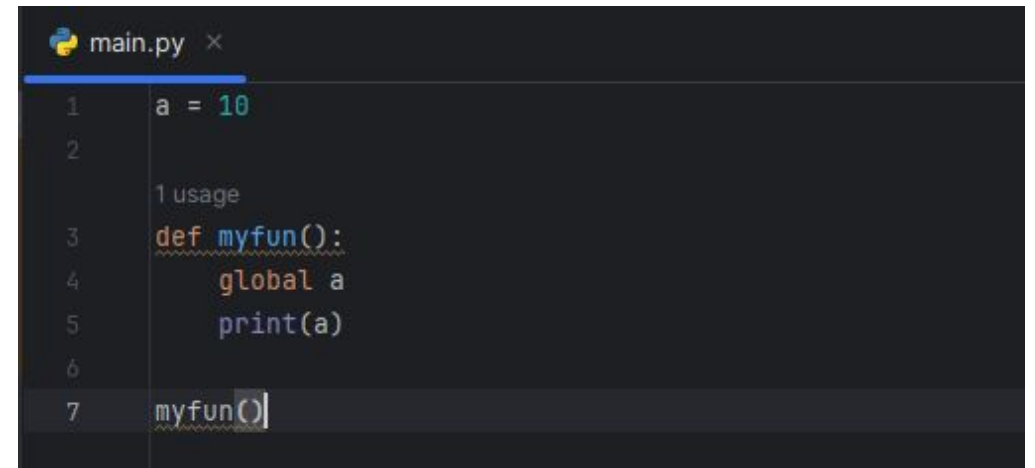


How are Variables Declared?

- Variables are declared by writing a line of code that includes the variable's name, data type, and value.
- Each programming language has its own conventions for declaring variables.
- Variables are usually confined to the block, function, or scope of declaration.

Global Variables

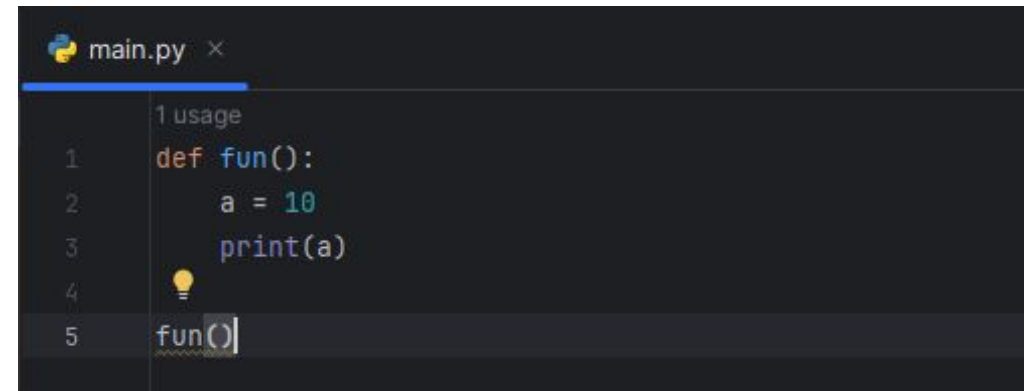
- Global variables are the types of variables that are declared outside of every function of the program.
- The global variable, in contrast to local variables, is accessible by all functions in a program.



```
main.py x
1  a = 10
2
   1 usage
3  def myfun():
4      global a
5      print(a)
6
7  myfun()
```

Local Variables

- Local variables are declared inside the function blocks. In Python, local variables can be declared at any location in the code block.
- Only statements that are written inside a function can access local variables. They are secure in the way that no other function or variable of that program can access them.

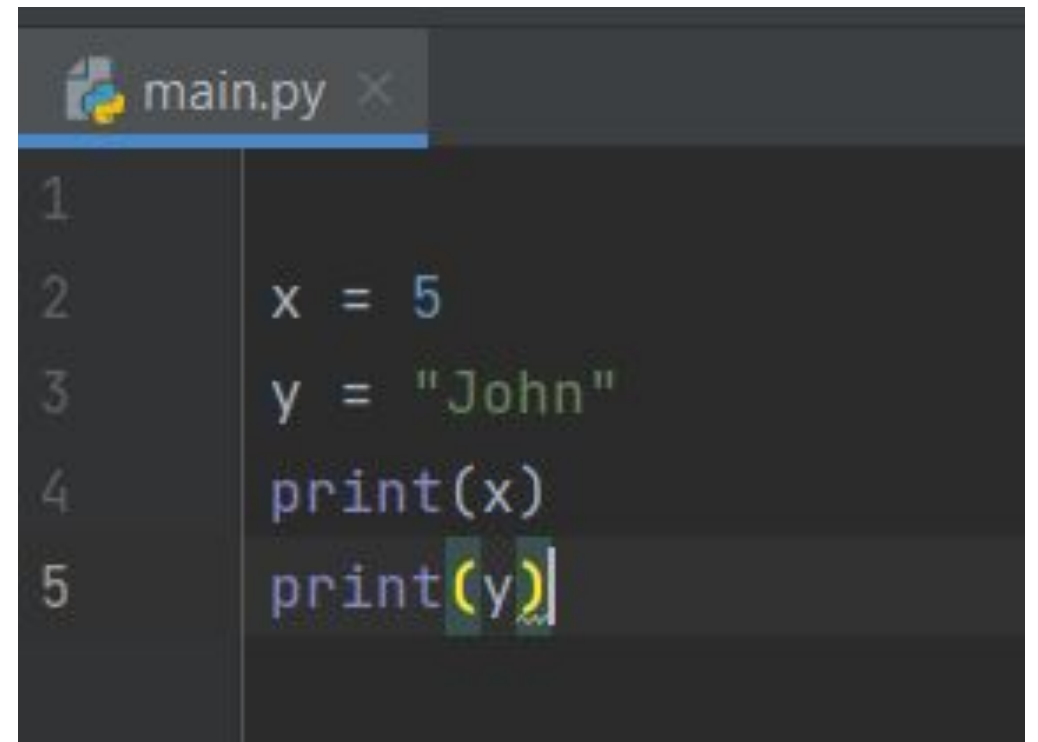


The screenshot shows a code editor window titled 'main.py'. The code defines a function 'fun()' with a local variable 'a' set to 10 and a 'print(a)' statement. The function is then called 'fun()' on line 5. A lightbulb icon is visible next to the function call, indicating a suggestion or tip.

```
1 usage
2 def fun():
3     a = 10
4     print(a)
5 fun()
```

Creating Variables

- Python has no command for declaring a variable.
- A variable is created the moment you first assign a value to it.
- Variables do not need to be declared with any particular *type*, and can even change type after they have been set.

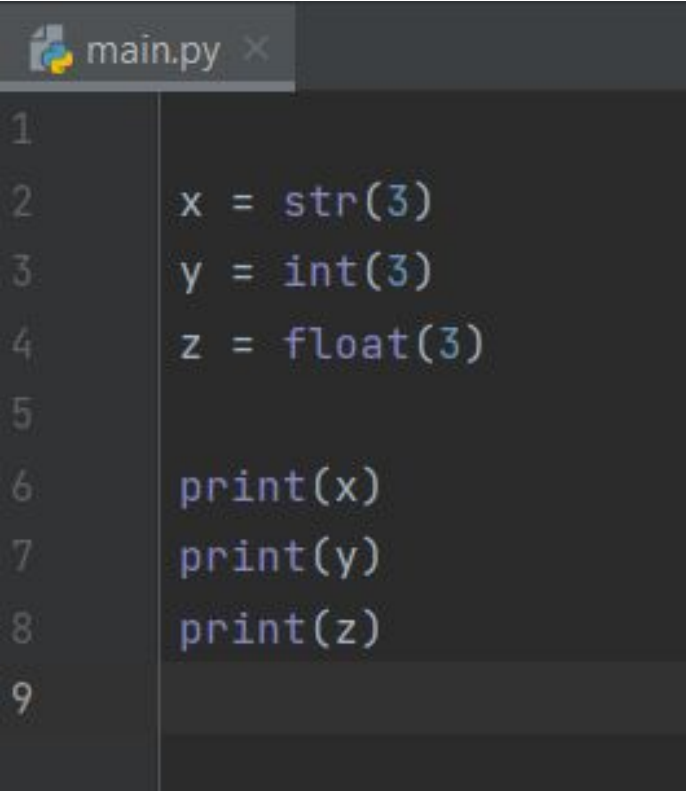
A screenshot of a Python IDE window titled 'main.py'. The code is as follows:

```
1  
2     x = 5  
3     y = "John"  
4     print(x)  
5     print(y)
```

The code demonstrates the creation of two variables, `x` and `y`, and their subsequent printing. `x` is assigned the integer value 5, and `y` is assigned the string value "John". The `print` statements output the values of these variables.

Casting Variable

- If you want to specify the data type of a variable, this can be done with casting.

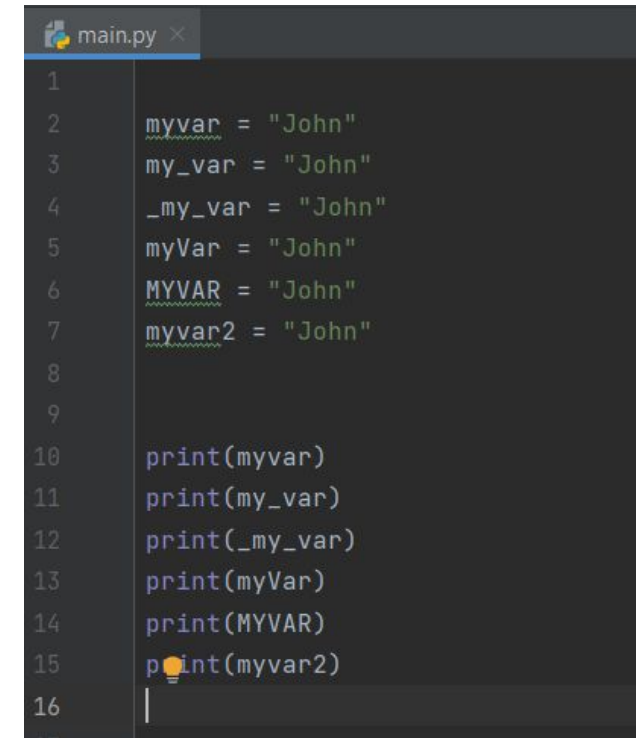


```
main.py x
1
2     x = str(3)
3     y = int(3)
4     z = float(3)
5
6     print(x)
7     print(y)
8     print(z)
9
```

The image shows a code editor window titled 'main.py'. It contains a Python script with 9 lines of code. Lines 2, 3, and 4 demonstrate casting: 'x' is cast to a string using 'str(3)', 'y' is cast to an integer using 'int(3)', and 'z' is cast to a float using 'float(3)'. Lines 6, 7, and 8 show the variables being printed. The code is syntax-highlighted with blue for keywords and black for identifiers and literals.

Variable Names

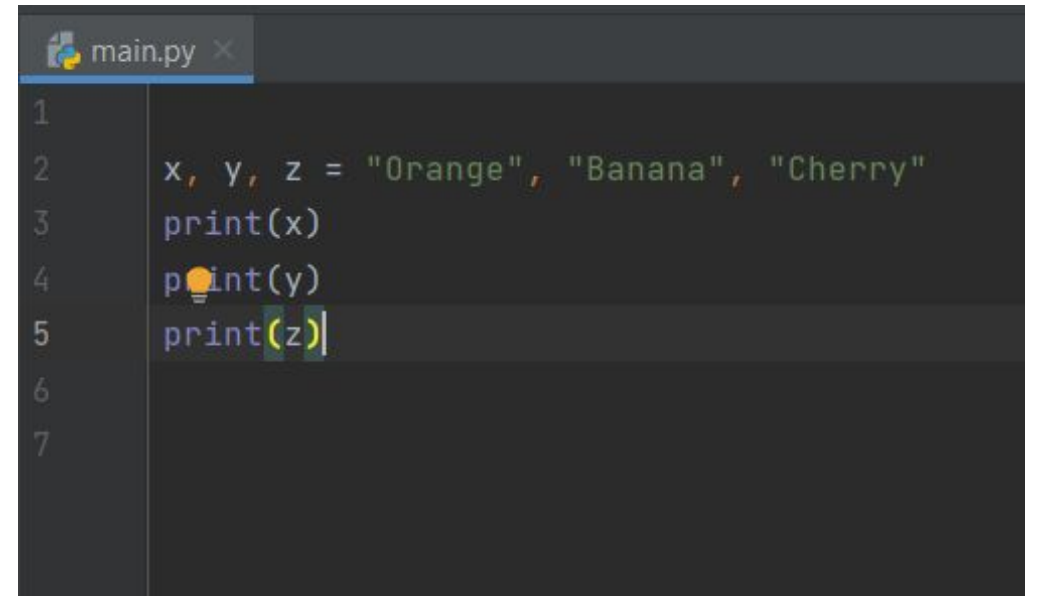
- A variable can have a short name (like x and y) or a more descriptive name (age, carname, total_volume).
Rules for Python variables: A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- Variable names are case-sensitive (age, Age and AGE are three different variables)



```
1  
2  myvar = "John"  
3  my_var = "John"  
4  _my_var = "John"  
5  myVar = "John"  
6  MYVAR = "John"  
7  myvar2 = "John"  
8  
9  
10 print(myvar)  
11 print(my_var)  
12 print(_my_var)  
13 print(myVar)  
14 print(MYVAR)  
15 print(myvar2)  
16 |
```

Many Values to Multiple Variables

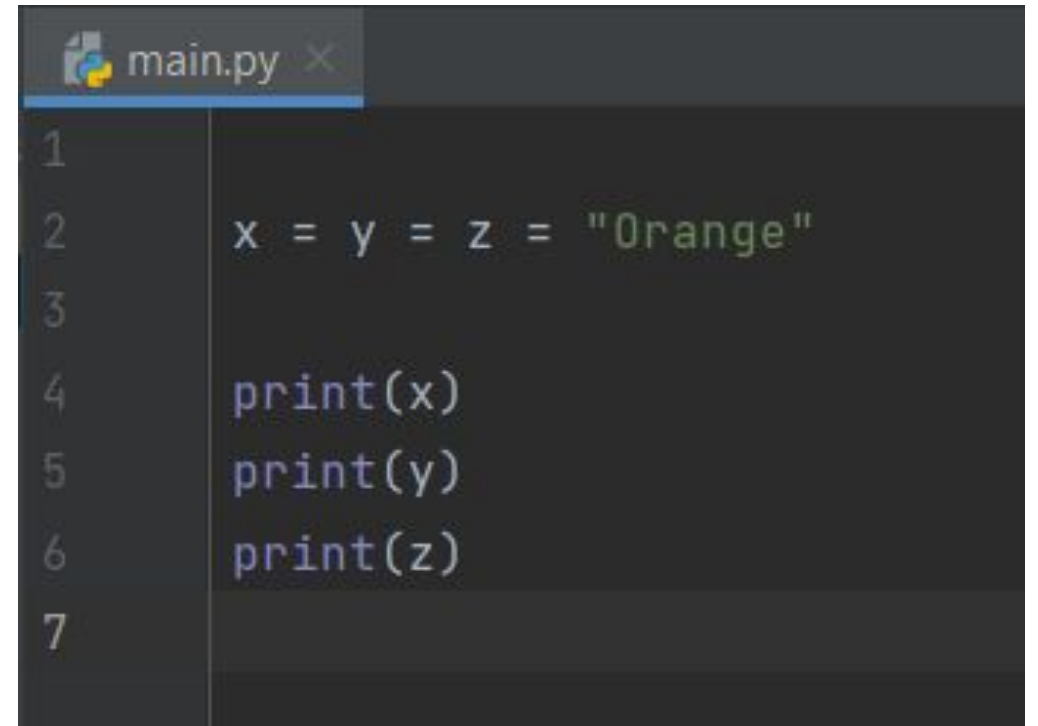
- Python allows you to assign values to multiple variables in one line.

A screenshot of a code editor window titled 'main.py'. The editor shows a Python script with five lines of code. Line 1 is empty. Line 2 assigns three string values to variables x, y, and z. Lines 3, 4, and 5 each print the value of x, y, and z respectively. Line 6 is empty and line 7 is partially visible.

```
1
2 x, y, z = "Orange", "Banana", "Cherry"
3 print(x)
4 print(y)
5 print(z)
6
7
```

One Value to Multiple Variables

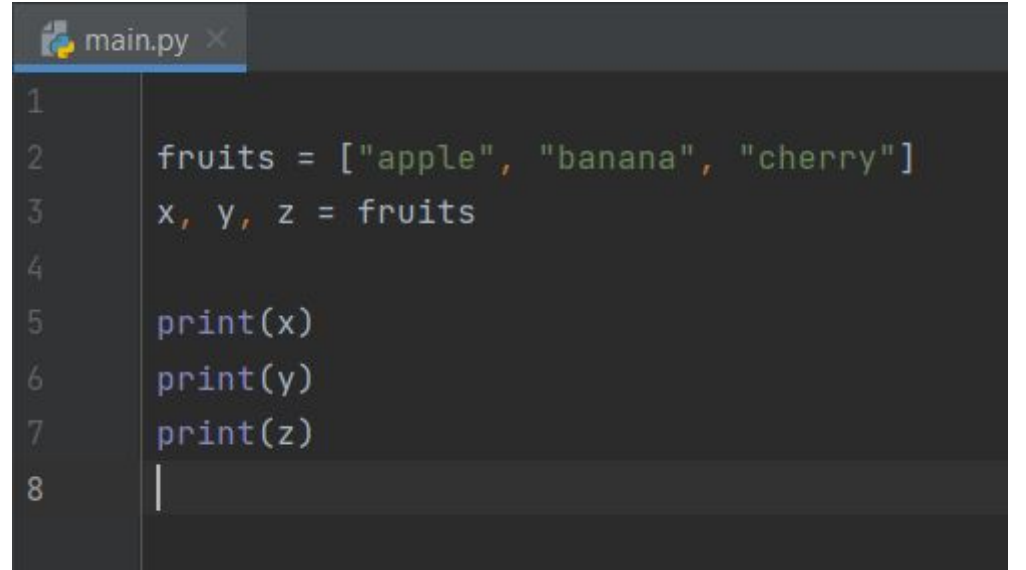
- And you can assign the *same* value to multiple variables in one line.



```
1  
2 x = y = z = "Orange"  
3  
4 print(x)  
5 print(y)  
6 print(z)  
7
```

Unpack a Collection

- If you have a collection of values in a list, tuple etc. Python allows you to extract the values into variables. This is called *unpacking*.

A screenshot of a code editor window titled 'main.py'. The editor shows a Python script with 8 lines of code. Line 2 defines a list 'fruits' containing 'apple', 'banana', and 'cherry'. Line 3 performs list unpacking by assigning 'x', 'y', and 'z' to the elements of 'fruits'. Lines 5, 6, and 7 print the values of 'x', 'y', and 'z' respectively. Line 8 is empty.

```
1  
2     fruits = ["apple", "banana", "cherry"]  
3     x, y, z = fruits  
4  
5     print(x)  
6     print(y)  
7     print(z)  
8     |
```




Basic Syntax on DATA Types

- 1. list**
 - 2. tuple**
 - 3. dict**
 - 4. set**
-

What is Data Types

- A **data type** represents a kind of value and determines what operations can be done on it. Numeric, non-numeric and Boolean (true/false) data are the most obvious data types.

Data Types in Python

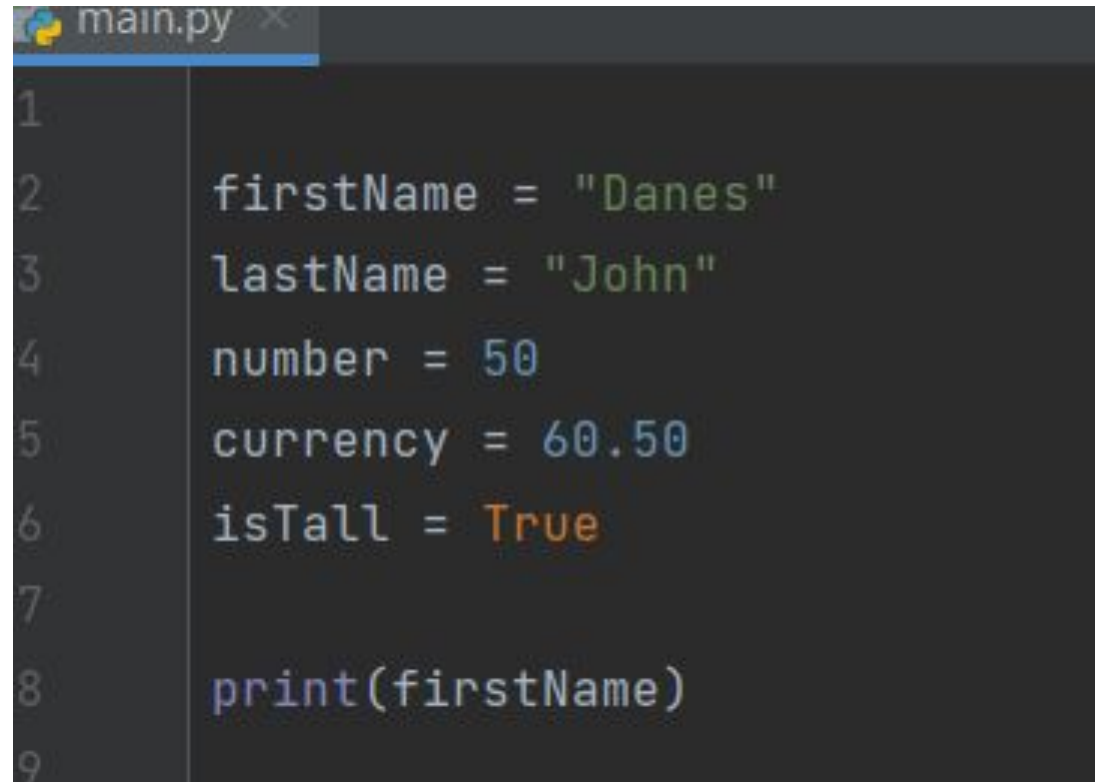
- Python Data Types are used to define the type of a variable.
- It defines what type of data we are going to store in a variable.
- The data stored in memory can be of many types.

Basic Data Types

- **string** (Sentence, Phrases, Characters): Identifier = "Hello World"
- **int** (Positive & Negative Numbers): Identifiers = 5
- **float** (Decimal Numbers): Identifiers = 25.3237
- **bool** (True or False): Identifier = true

print() Variable

- Used to display something in the console.



```
main.py x
1
2     firstName = "Danes"
3     lastName = "John"
4     number = 50
5     currency = 60.50
6     isTall = True
7
8     print(firstName)
9
```

Casting Variables

- A technique used to convert a datatype to another datatype.

Syntax:

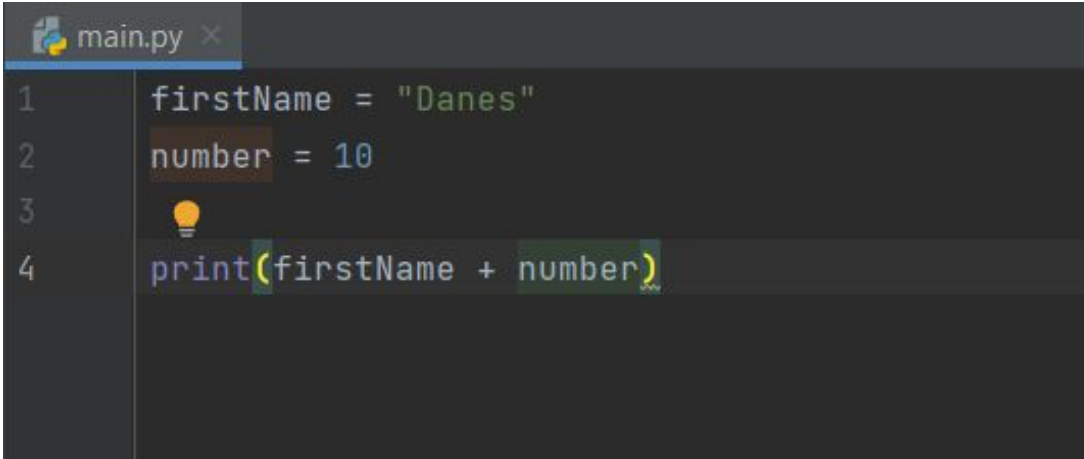
Convert Numbers to String

`str(number)`

Convert String to Number

`int(string)`

`float(string)`



```
main.py x
1  firstName = "Danes"
2  number = 10
3
4  print(firstName + number)
```

Arithmetic Operators

- Used to perform mathematical operations inside our programming language.

Operator	Syntax	Result
Subtraction	$x - y$	Difference
Addition	$x + y$	Sum
Multiplication	$x * y$	Product
Division	x / y	Quotient
Modulus	$x \% y$	Remainder
Floor Division	$x // y$	Quotient (rounded off)
Exponent	$x ** y$	Power

One Function Calculator

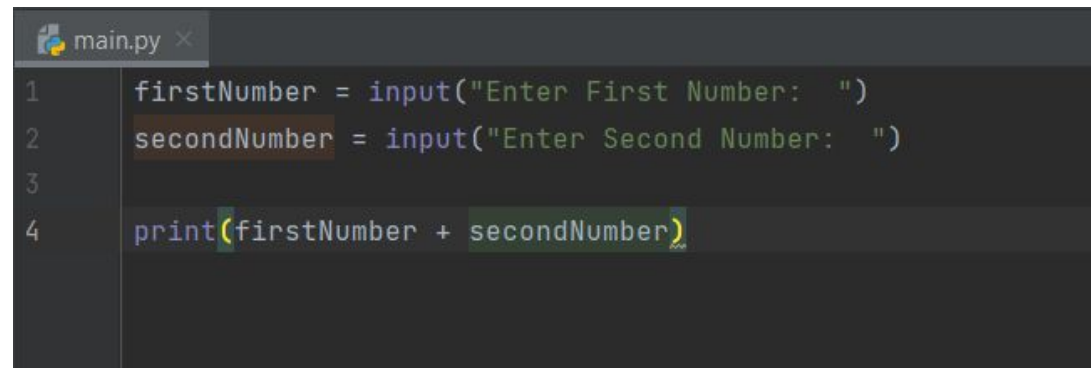
- Try to create a program that will perform one of the arithmetic operators when the user inputs 2 numbers

Sample Output:

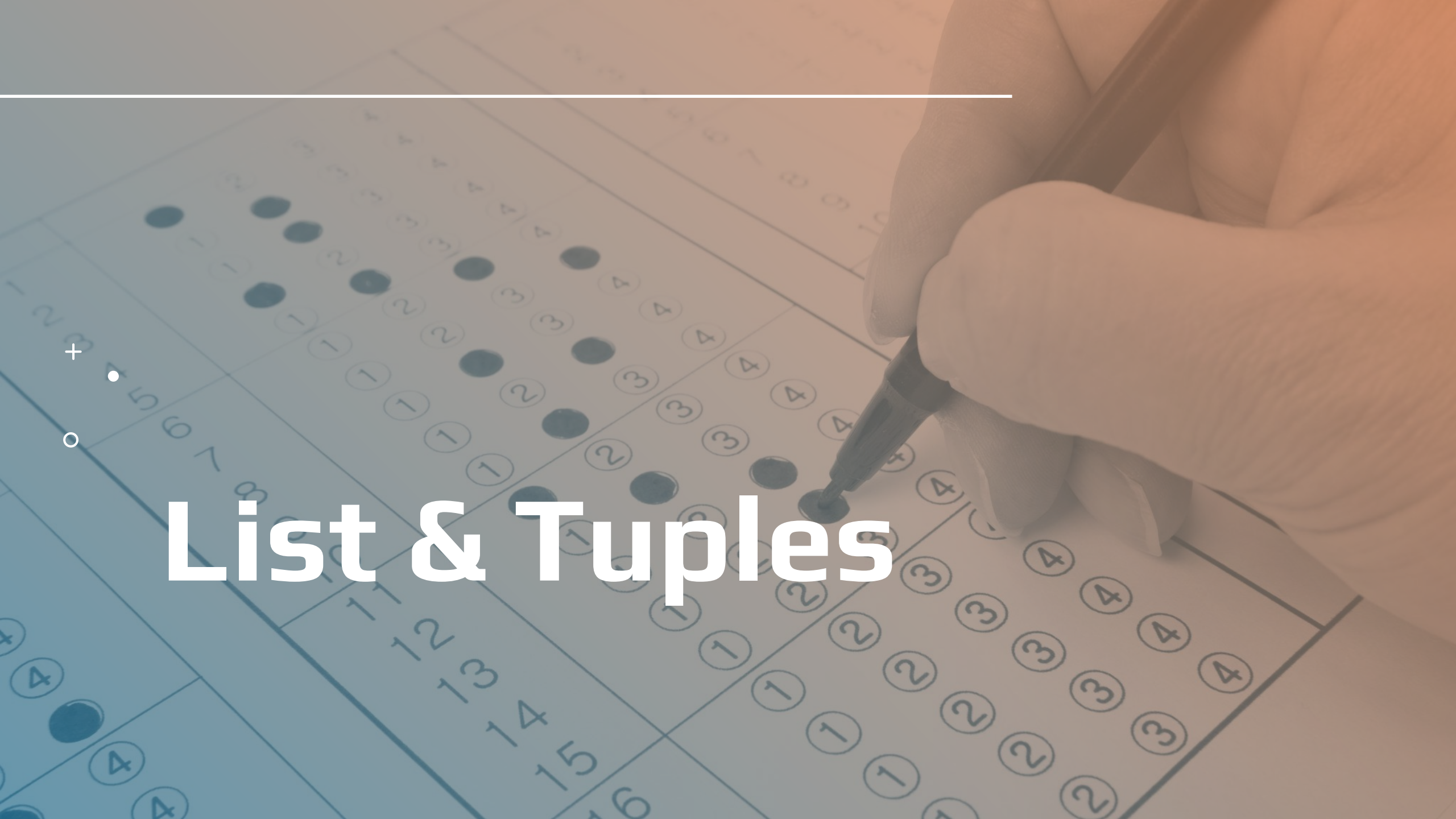
First Number: 25

Second Number: 20

25 – 20 = 5

A screenshot of a code editor window titled 'main.py'. The editor contains four lines of Python code. Line 1: 'firstNumber = input("Enter First Number: ")'. Line 2: 'secondNumber = input("Enter Second Number: ")'. Line 3: (empty). Line 4: 'print(firstNumber + secondNumber)'. The code is syntax-highlighted with colors: strings are green, keywords are blue, and identifiers are white. The background is dark grey.

```
main.py x
1 firstNumber = input("Enter First Number: ")
2 secondNumber = input("Enter Second Number: ")
3
4 print(firstNumber + secondNumber)
```

List & Tuples

+

•

○

What is List?



Lists are used to store multiple items in a single variable.



Lists are one of 4 built-in data types in Python used to store collections of data, the other 3 are Tuple, Set, and Dictionary, all with different qualities and usage.



Lists are created using square brackets.



Lists are just like dynamically sized arrays, declared in other languages



In simple language, a list is a collection of things, enclosed in [] and separated by commas.

What is Tuples?



Python tuple is another sequence data type that is similar to a list. A Python tuple consists of a number of values separated by commas. Unlike lists, however, tuples are enclosed within parentheses (...).



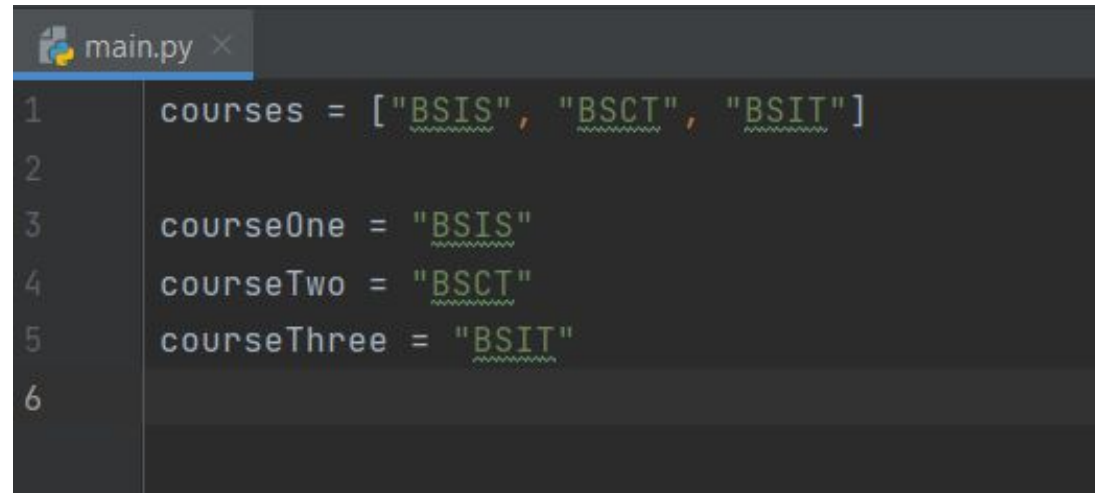
Tuple is one of the built-in data types in Python. A Python tuple is a sequence of comma separated items, enclosed in parentheses ().



The items in a Python tuple need not be of same data type.

Lists

- A Read and Write Collection of Variables that may be used to sort certain data.
 - *Reading Whole Lists*
 - *Reading List ITEMS*



```
main.py x
1  courses = ["BSIS", "BSCT", "BSIT"]
2
3  courseOne = "BSIS"
4  courseTwo = "BSCT"
5  courseThree = "BSIT"
6
```

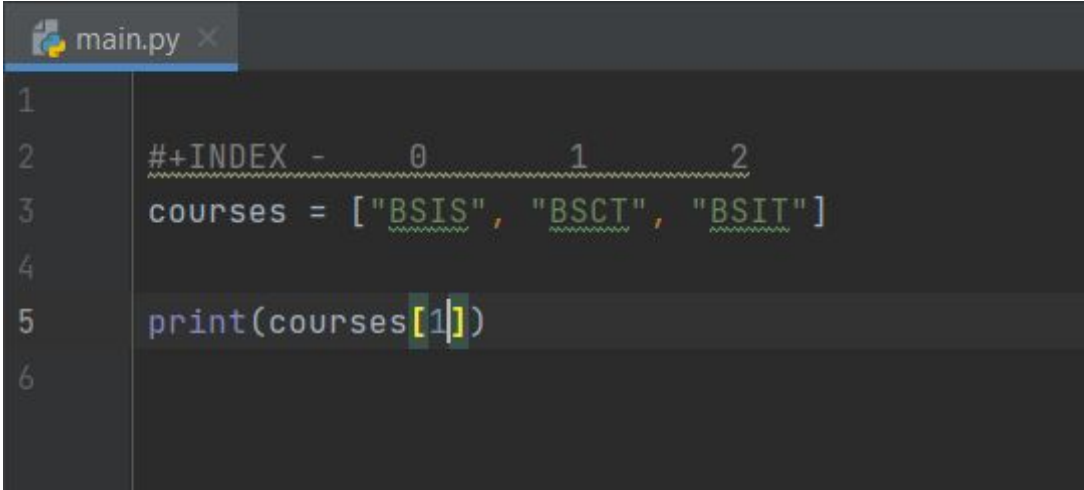
Index

- The number of where an item is on a collection.

+ INDEX: 0 1 2

- INDEX: -3 -2 -1

`courses = ["BSIS", "BSCT", "BSIT"]`



```
main.py x
1
2  #+INDEX - 0      1      2
3  courses = ["BSIS", "BSCT", "BSIT"]
4
5  print(courses[1])
6
```

The screenshot shows a code editor window titled 'main.py'. It contains a Python list named 'courses' with three elements: 'BSIS', 'BSCT', and 'BSIT'. Above the list, a comment line shows the positive indices 0, 1, and 2 aligned with the elements. Below the list, a print statement is shown with the index 1 highlighted, indicating it will output the second element, 'BSCT'.

Reading Lists RANGE

- You can read a list's range of items by specifying a range of index

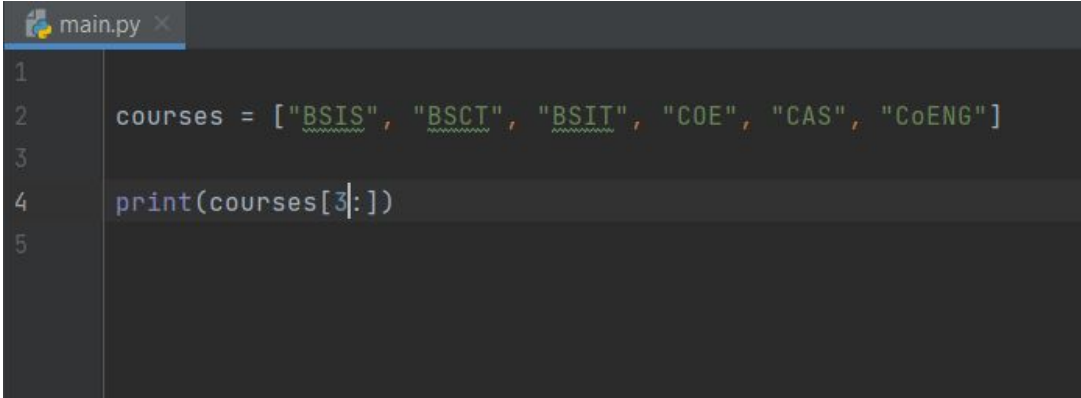
Syntax

`print(list[startIndex: endIndex])`

`print(list[:endIndex])`

`print(list[startIndex:])`

NOTE: endIndex Item is excluded.



```
main.py x
1
2  courses = ["BSIS", "BSCT", "BSIT", "COE", "CAS", "CoENG"]
3
4  print(courses[3:])
5
```

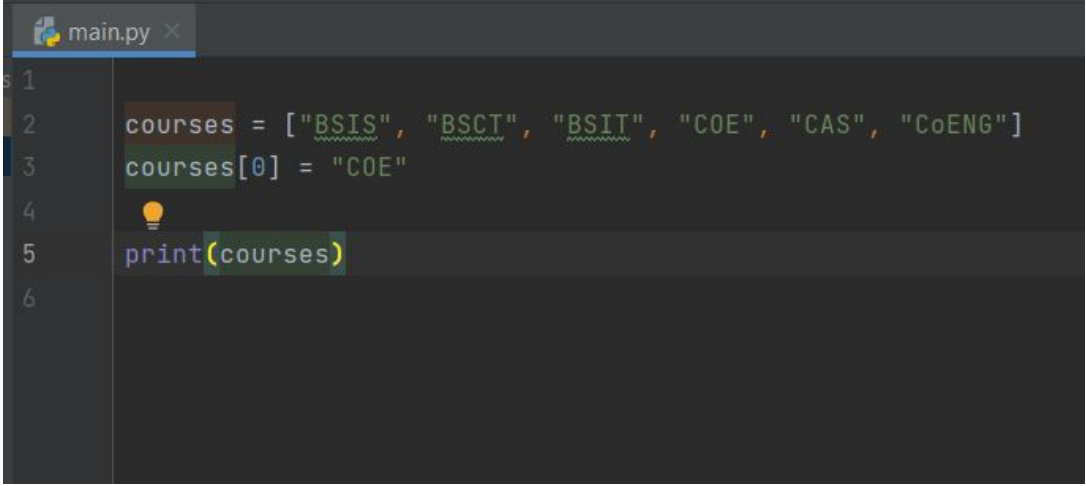
Assigning List ITEMS

- You can assign a list item by using an INDEX and an Assignment Operator “=”

Syntax

list[index] = value

list[0] = “COE”

A screenshot of a Python IDE window titled 'main.py'. The code is as follows:

```
1  
2  courses = ["BSIS", "BSCT", "BSIT", "COE", "CAS", "CoENG"]  
3  courses[0] = "COE"  
4  
5  print(courses)  
6
```

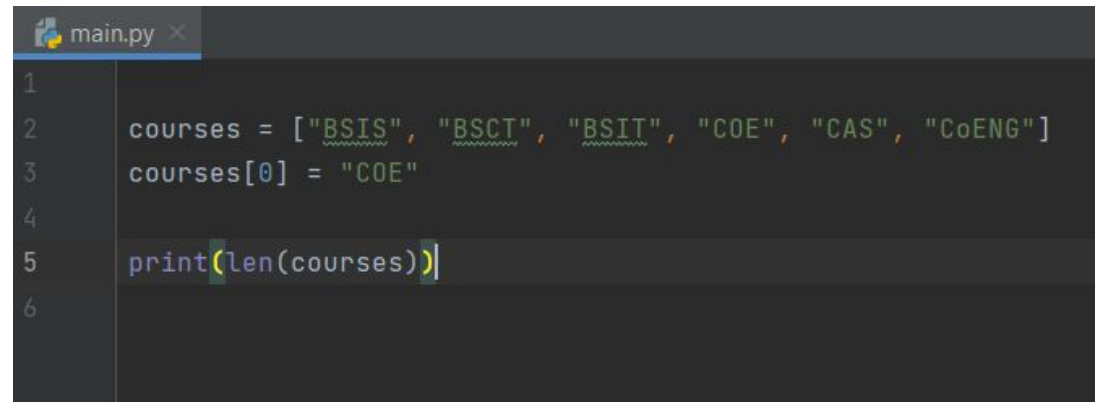
The code demonstrates creating a list of course codes and then updating the first element at index 0 to "COE".

List LENGTH

- You can check the number of items in a list by using the len() function

Syntax

len(list)



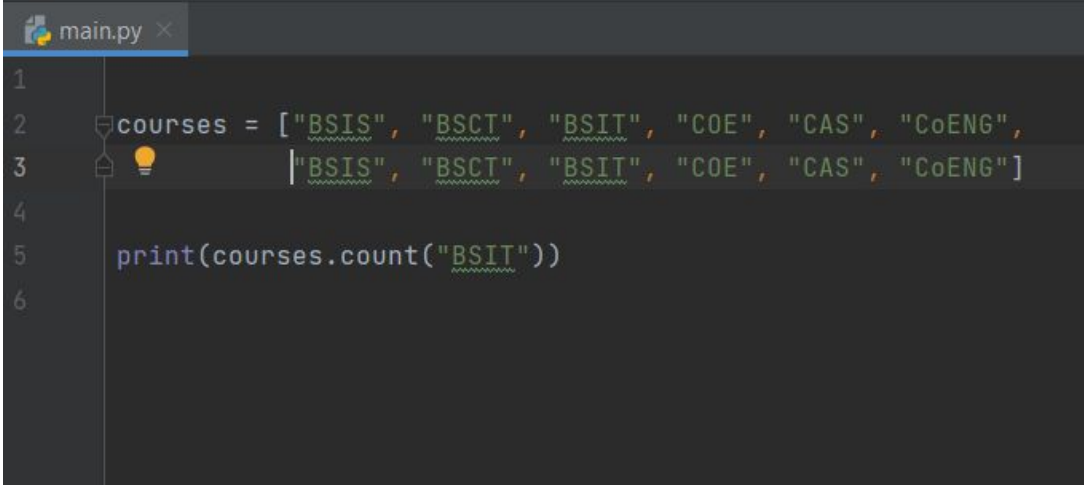
```
main.py x
1
2 courses = ["BSIS", "BSCT", "BSIT", "COE", "CAS", "CoENG"]
3 courses[0] = "COE"
4
5 print(len(courses))
6
```


List Count

- You can count how many times an item occurs in a list by using the `count()` function.

Syntax

list.count(value)



```
main.py x
1
2 courses = ["BSIS", "BSCT", "BSIT", "COE", "CAS", "CoENG",
3           "BSIS", "BSCT", "BSIT", "COE", "CAS", "CoENG"]
4
5 print(courses.count("BSIT"))
6
```

The screenshot shows a Python IDE window titled 'main.py'. The code defines a list named 'courses' containing 12 elements: 'BSIS', 'BSCT', 'BSIT', 'COE', 'CAS', 'CoENG', 'BSIS', 'BSCT', 'BSIT', 'COE', 'CAS', and 'CoENG'. The list is printed on two lines. On the third line, there is a lightbulb icon indicating a suggestion or tip. The code then prints the count of 'BSIT' in the list using the `count()` method.

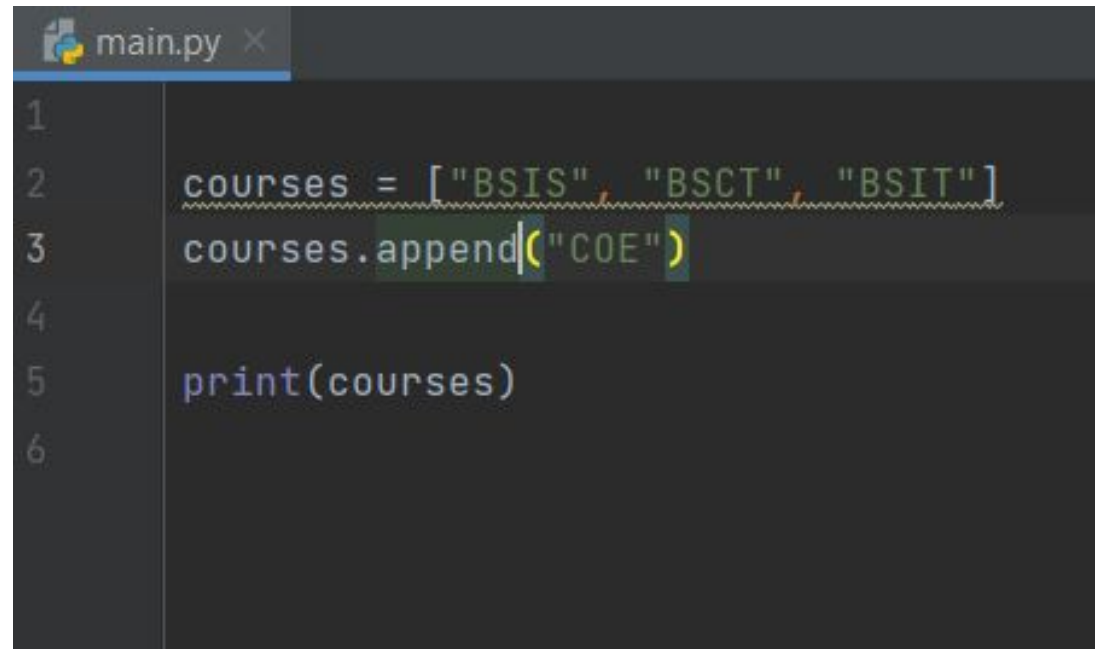
List ADD ITEMS by APPEND()

- `append()` adds an item at the END OFF THE LIST.

Syntax

list.append(value)

list.append("COE")



```
main.py x
1
2  courses = ["BSIS", "BSCT", "BSIT"]
3  courses.append("COE")
4
5  print(courses)
6
```

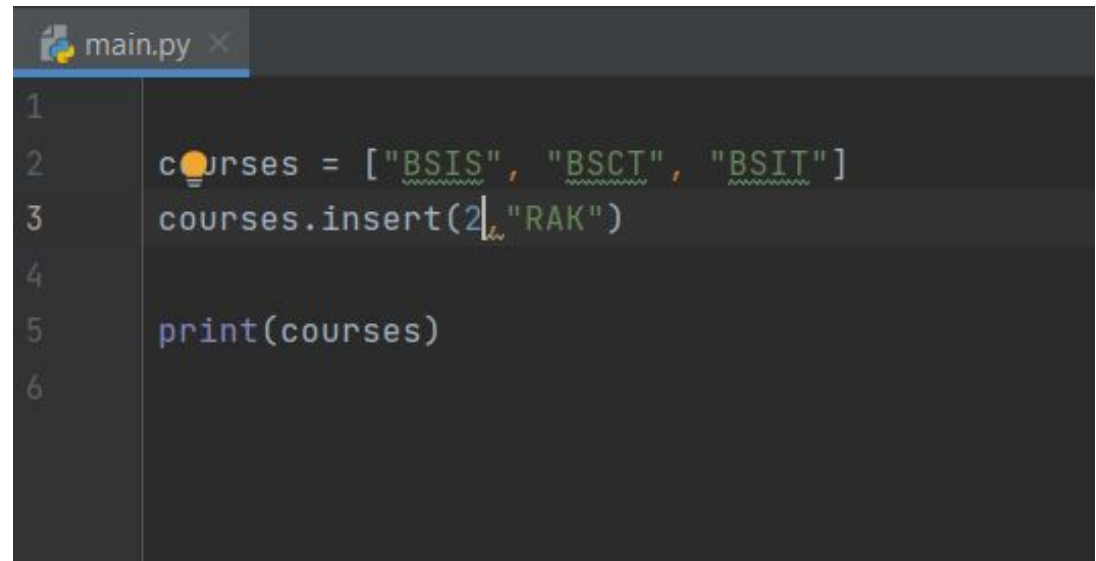
List ADD ITEMS by INSERT()

- insert() adds an item at the specified INDEX.

Syntax

list.insert(index,value)

list.insert(0,"RAK")



```
main.py x
1
2  courses = ["BSIS", "BSCT", "BSIT"]
3  courses.insert(2, "RAK")
4
5  print(courses)
6
```

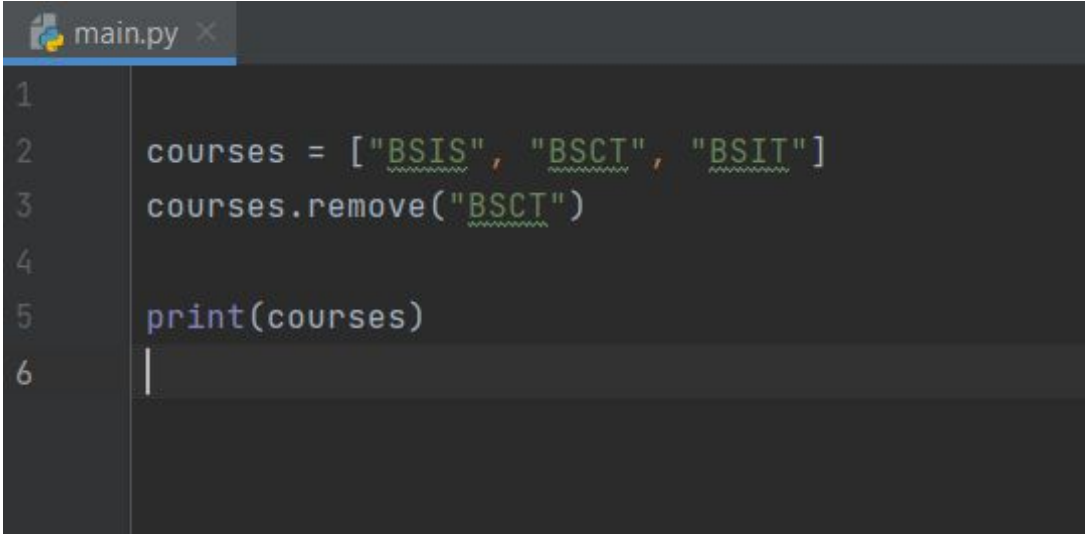
List DELETING ITEMS by REMOVE()

- `remove()` deletes an item based on their value.

Syntax

`list.remove(value)`

`List.remove("RAK")`

A screenshot of a Python IDE window titled 'main.py'. The code is as follows:

```
1  
2   courses = ["BSIS", "BSCT", "BSIT"]  
3   courses.remove("BSCT")  
4  
5   print(courses)  
6   |
```

The code demonstrates creating a list 'courses' with three elements: 'BSIS', 'BSCT', and 'BSIT'. It then uses the 'remove()' method to delete the element 'BSCT'. Finally, it prints the list, which would output ['BSIS', 'BSIT'].

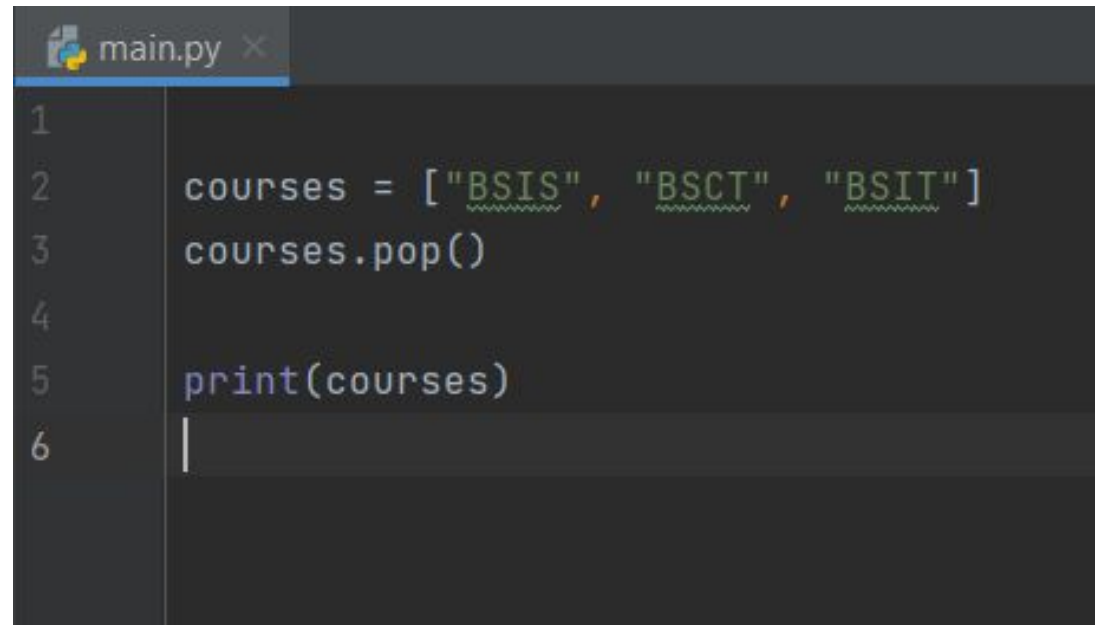
List DELETING ITEMS by POP()

- pop() deletes an item based on their index but if index is not specified it deletes the last item.

Syntax

list.pop()

list.pop(index)



```
main.py x
1
2  courses = ["BSIS", "BSCT", "BSIT"]
3  courses.pop()
4
5  print(courses)
6  |
```

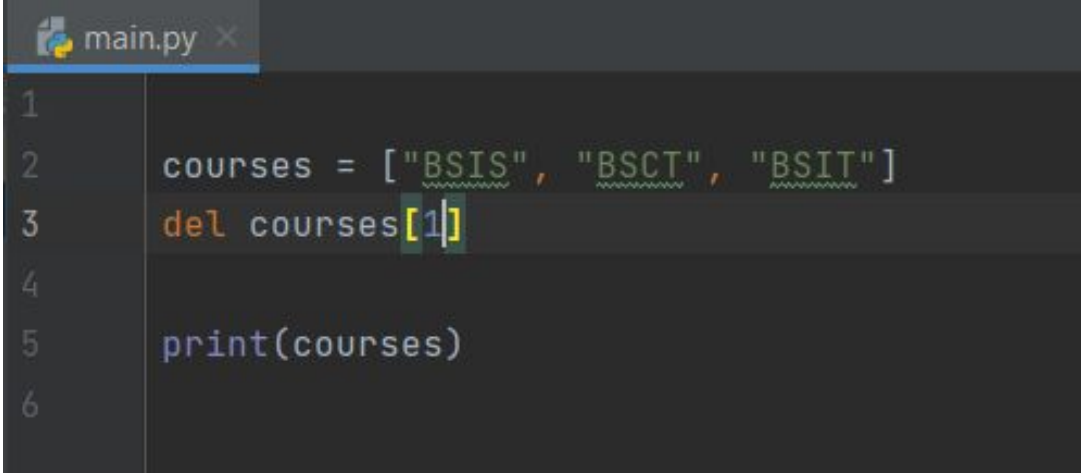
List DELELTING ITEMS by DEL

- del deletes an item based on their index but if index is not specified it deletes the whole list

Syntax

del list[index]

del list

A screenshot of a Python IDE window titled 'main.py'. The code is as follows:

```
1  
2  courses = ["BSIS", "BSCT", "BSIT"]  
3  del courses[1]  
4  
5  print(courses)  
6
```

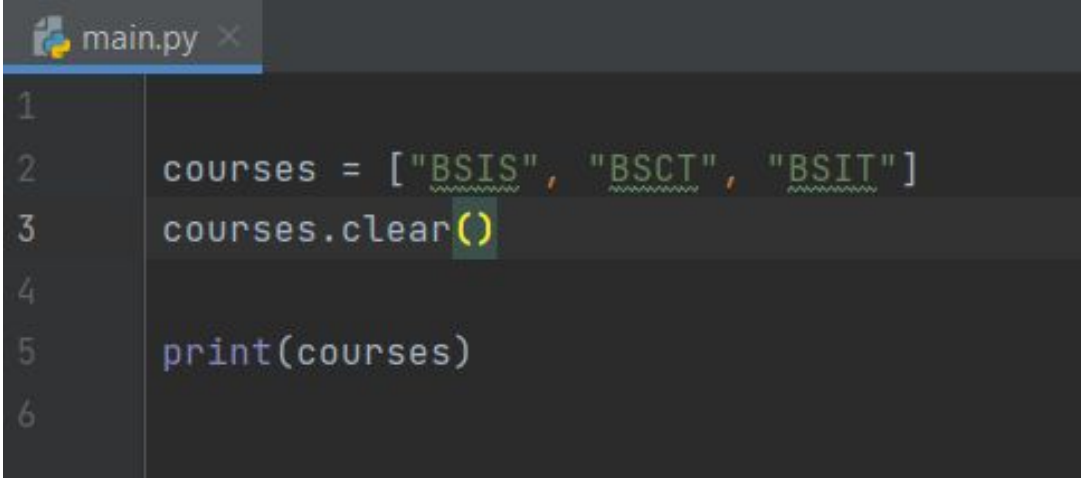
The code demonstrates creating a list 'courses' with three elements: "BSIS", "BSCT", and "BSIT". Then, the second element at index 1 ("BSCT") is deleted using the 'del' statement. Finally, the list is printed to show the remaining elements.

Clearing a List

- `clear()` deletes all the value in a list

Syntax

`list.clear()`



The screenshot shows a code editor window titled 'main.py'. The code is as follows:

```
1
2  courses = ["BSIS", "BSCT", "BSIT"]
3  courses.clear()
4
5  print(courses)
6
```

The code demonstrates the use of the `clear()` method on a list named `courses`. The list initially contains the strings `"BSIS"`, `"BSCT"`, and `"BSIT"`. After calling `courses.clear()`, the list is emptied, and `print(courses)` will output an empty list.

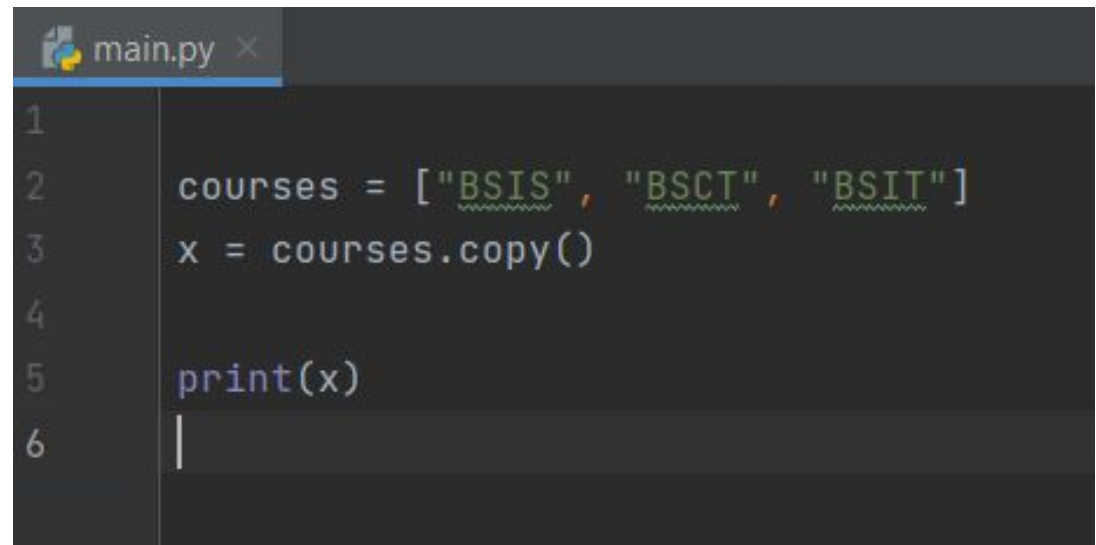
Copying a List

- `copy()` copies the whole list which can be assigned to a new list.

Syntax

```
listOne = ["BSIS", "BSCT", "BSIT"]
```

```
listTwo = listOne.copy()
```

A screenshot of a Python IDE window titled 'main.py'. The code is as follows:

```
1  
2     courses = ["BSIS", "BSCT", "BSIT"]  
3     x = courses.copy()  
4  
5     print(x)  
6     |
```

The strings "BSIS", "BSCT", and "BSIT" are underlined with green dashed lines, indicating they are recognized as string literals. The variable 'x' is assigned the result of 'courses.copy()'. The cursor is at the end of line 6.

COMBINING Lists BY ADDING

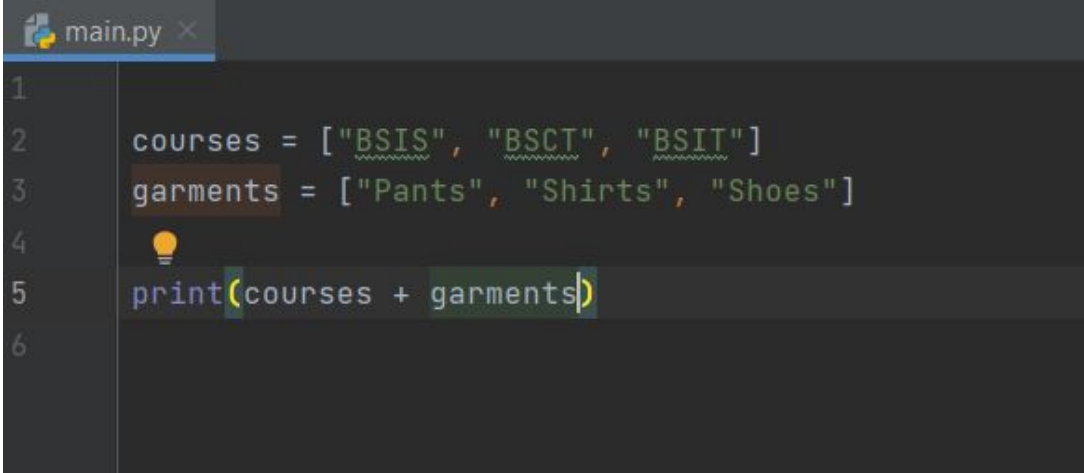
- You can use '+' operator to combine lists.

Syntax

listOne = ["BSIS", "BSCT", "BSIT"]

listTwo = ["Pants", "Shirt", "Shoes"]

listThree = *listOne* + *listTwo*



The screenshot shows a code editor window titled 'main.py'. It contains the following Python code:

```
1
2  courses = ["BSIS", "BSCT", "BSIT"]
3  garments = ["Pants", "Shirts", "Shoes"]
4
5  print(courses + garments)
6
```

A lightbulb icon is visible next to line 4, indicating a tip or suggestion.

COMBINING Lists BY EXTEND()

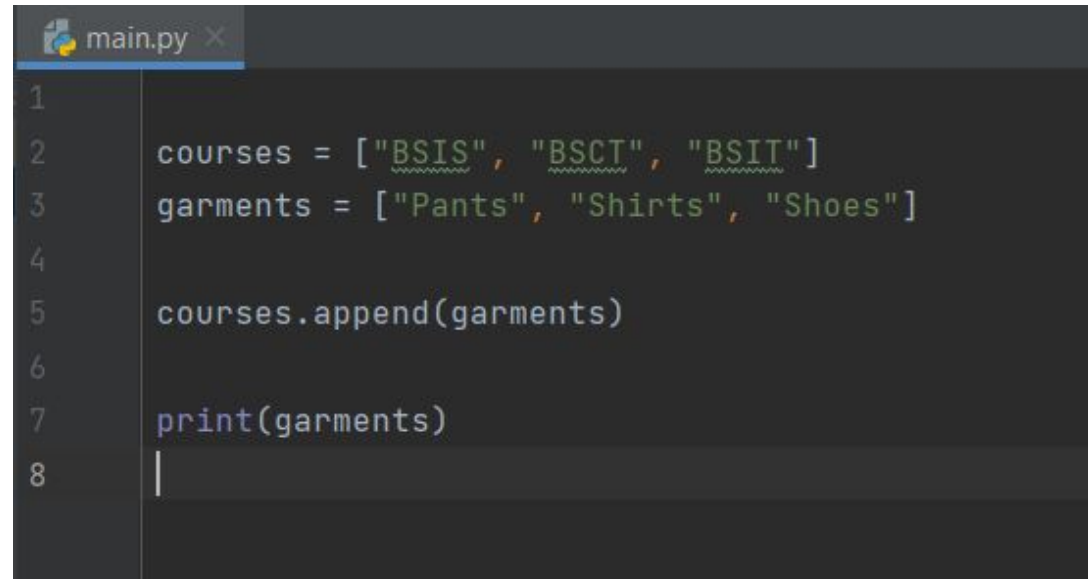
- `extend()` combines list by appending the specified list to the of the first list.

Syntax

listOne = ["BSIS", "BSCT", "BSIT"]

listTwo = ["Pants", "Shirt", "Shoes"]

listOne.extend(listTwo)



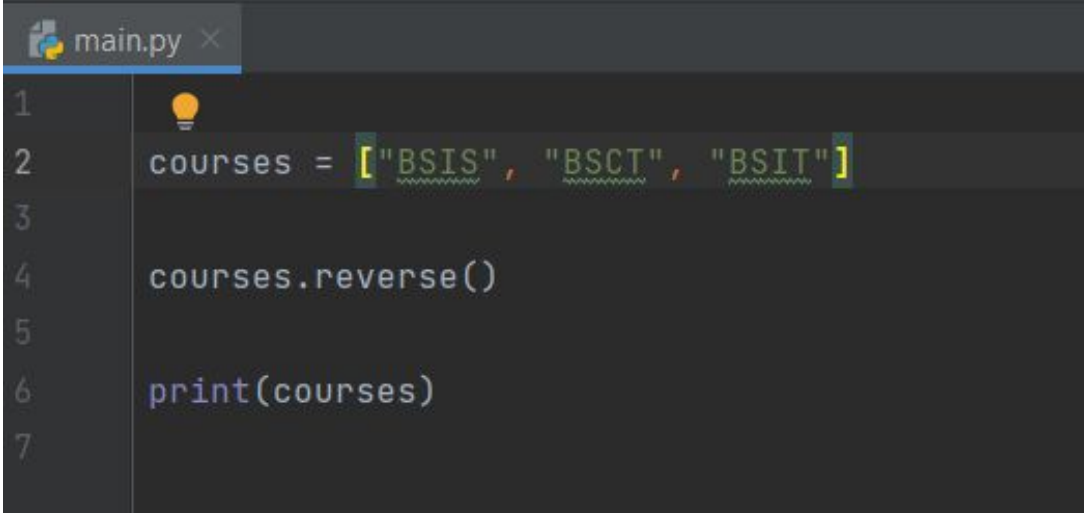
```
1
2  courses = ["BSIS", "BSCT", "BSIT"]
3  garments = ["Pants", "Shirts", "Shoes"]
4
5  courses.append(garments)
6
7  print(garments)
8  |
```

REVERSE Lists Items

- `reverse()` Reverse the order of the List's Items.

Syntax

`list.reverse()`

A screenshot of a Python IDE window titled 'main.py'. The code is as follows:

```
1  
2  courses = ["BSIS", "BSCT", "BSIT"]  
3  
4  courses.reverse()  
5  
6  print(courses)  
7
```

The list items are underlined with wavy lines, and the `reverse()` method call is highlighted.

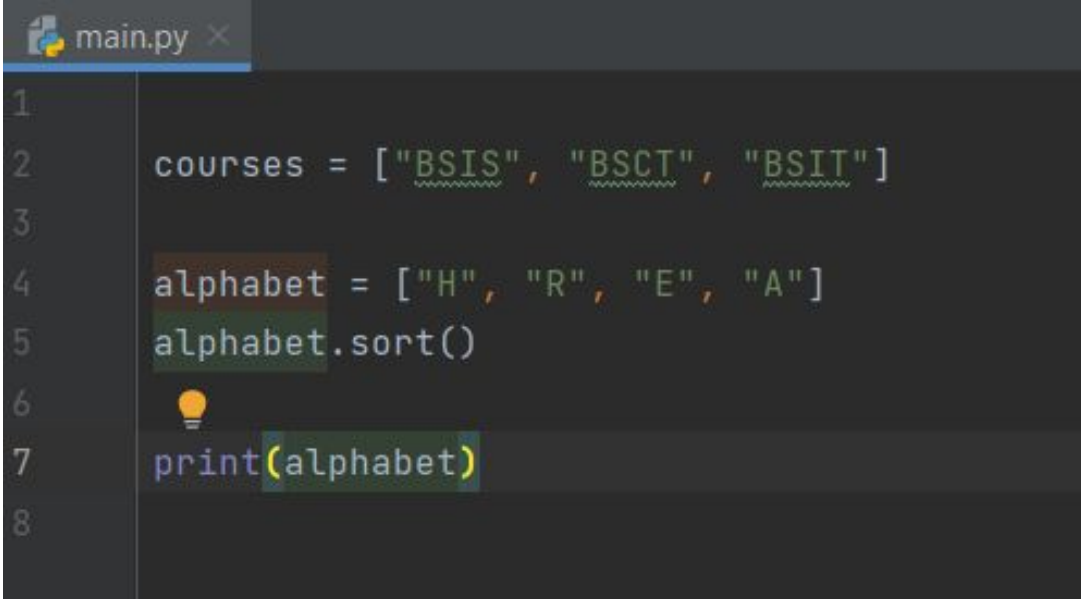
SORT Lists Items

- `sort()` Sort's List's Items by Alphabet or Value depending on the datatype.

Syntax

`list.sort()`#Ascending Order

`list.sort(reverse=True)`#Descending Order

A screenshot of a Python IDE window titled 'main.py'. The code is as follows:

```
1  
2     courses = ["BSIS", "BSCT", "BSIT"]  
3  
4     alphabet = ["H", "R", "E", "A"]  
5     alphabet.sort()  
6  
7     print(alphabet)  
8
```

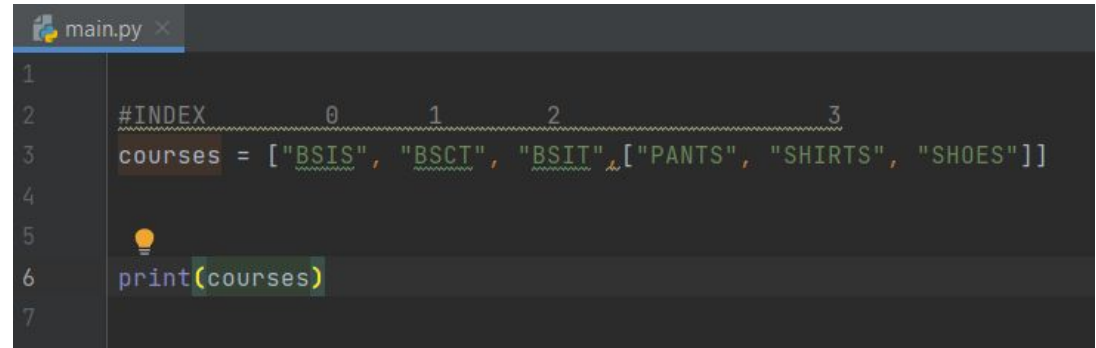
The code demonstrates sorting two lists: 'courses' and 'alphabet'. The 'courses' list contains ["BSIS", "BSCT", "BSIT"] and the 'alphabet' list contains ["H", "R", "E", "A"]. The 'alphabet' list is sorted in ascending order, resulting in ["A", "E", "H", "R"]. The IDE has a dark theme and a light blue header bar.

NESTED Lists

- A List inside a List also known as sublist

Syntax

```
course = ["BSIS", "BSCT",  
"BSIT", ["PANTS", "SHIRT", "SHOES"]]
```



```
main.py x
1
2 #INDEX      0      1      2      3
3 courses = ["BSIS", "BSCT", "BSIT", ["PANTS", "SHIRTS", "SHOES"]]
4
5
6 print(courses)
7
```

Tuples

- A Read-ONLY Collection of variables that may be used to sort certain data.

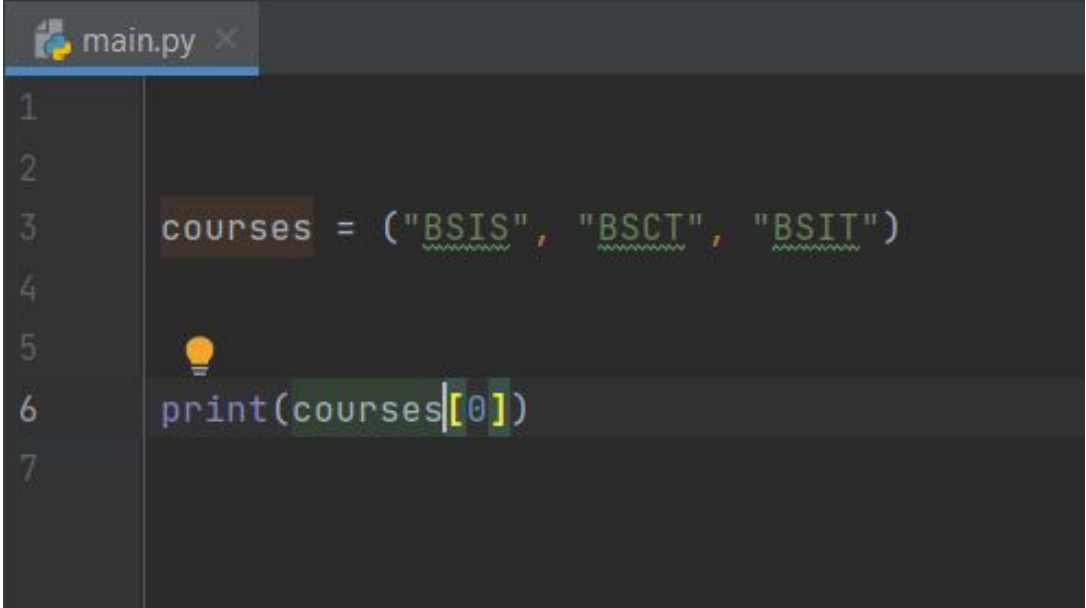
Syntax

identifier = (value, value1, value2)

courses = ("BSIS", "BSCT", "BSIT")

Tuples

- CAN be READ
- CAN be COMBINED
- CAN be DELETING COMPLETELY
- CAN'T be ASSIGNED
- CAN'T be DELETED ONE BY ONE



```
main.py x
1
2
3 courses = ("BSIS", "BSCT", "BSIT")
4
5
6 print(courses[0])
7
```

The screenshot shows a code editor window titled 'main.py'. It contains a Python script with two lines of code. Line 3 defines a tuple named 'courses' with three string elements: 'BSIS', 'BSCT', and 'BSIT'. Line 6 prints the first element of the tuple using indexing: 'print(courses[0])'. The code is syntax-highlighted, and a lightbulb icon is visible above the print statement, indicating a suggestion or tip.

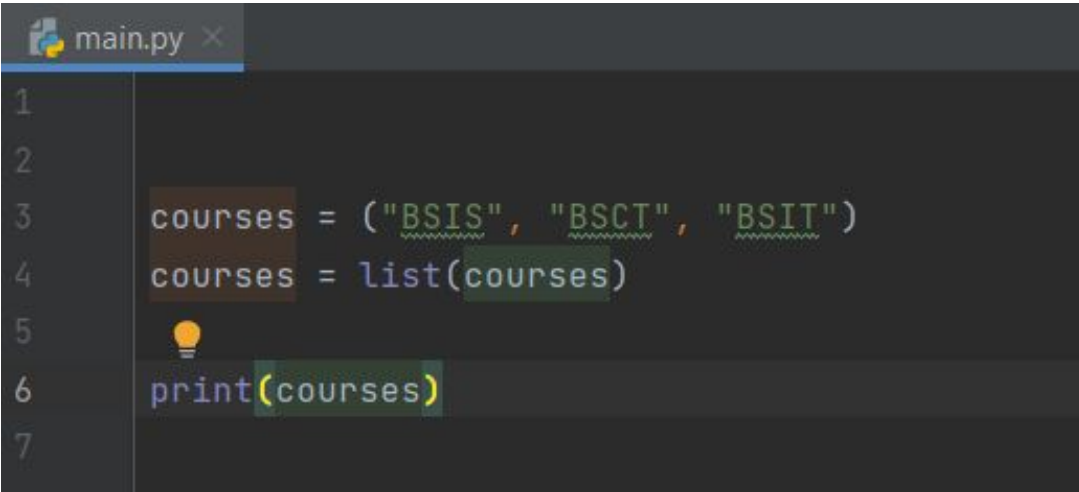
Casting TUPLES and LISTS

- Convert List to Tuple

tuple(list)

- Convert Tuple to List

list(tuple)



```
main.py x
1
2
3  courses = ("BSIS", "BSCT", "BSIT")
4  courses = list(courses)
5  
6  print(courses)
7
```

The screenshot shows a code editor window titled 'main.py'. It contains a Python script with 7 lines. Line 3 defines a tuple 'courses' with three elements: 'BSIS', 'BSCT', and 'BSIT'. Line 4 converts this tuple into a list using the 'list()' function. Line 6 prints the resulting list. A lightbulb icon is present on line 5, indicating a suggestion or tip.

Sets FUNCTION



What is Sets FUNCTION

The set() function creates a set object. The items in a set list are unordered, so it will appear in random order.

A set is a collection which is *unordered*, *unchangeable**, and *unindexed*.

Sets are used to store multiple items in a single variable.



Different aspects on Sets

- Understanding Sets
 - Manipulating Sets
-



Sets

- A Collection of Variables that is Partially Writable and its unordered and unindexed

Syntax

identifier = {*value*, *value1*, *value2*}

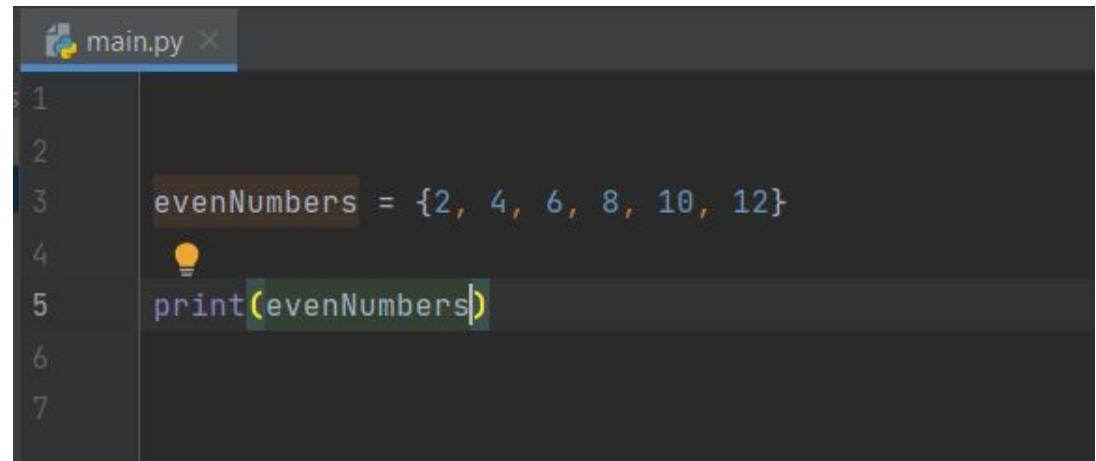
courses = {"BSIS", "BSCT", "BSIT"}

Reading the WHOLE Sets

- You can read a set by printing the whole set.

Syntax

print(set)

A screenshot of a Python IDE window titled 'main.py'. The code is as follows:

```
1  
2  
3 evenNumbers = {2, 4, 6, 8, 10, 12}  
4  
5 print(evenNumbers)  
6  
7
```

A lightbulb icon is visible next to line 4, and the 'print(evenNumbers)' statement on line 5 is highlighted with a green background.

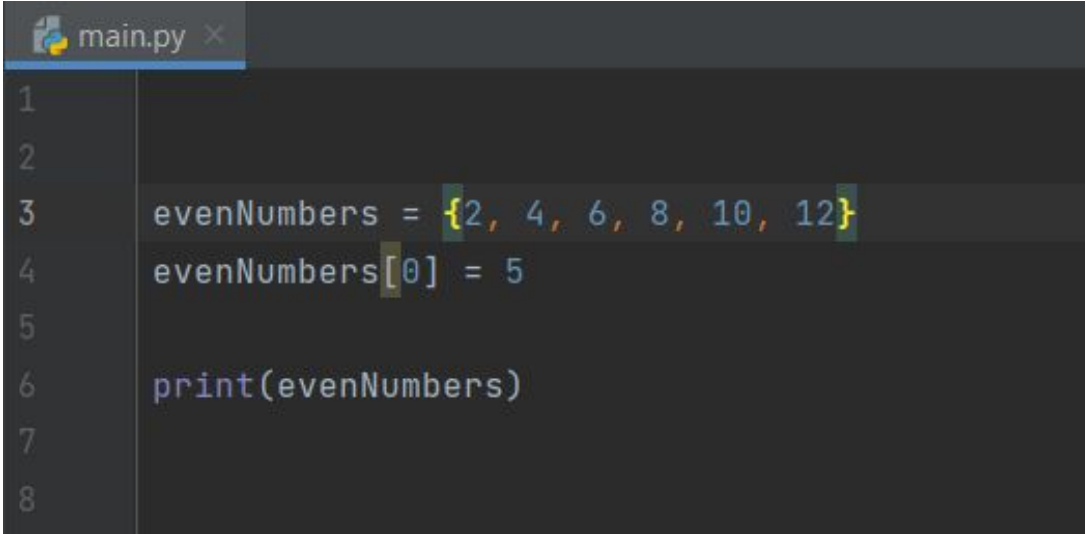


Reading Set ITEMS

- It is not possible to read certain item in a set unless you cast it into a List or Tuple
 - Hence Sets are unindexed and unordered
-

Assigning Set ITEMS

- It is not possible to change the value of a certain item in a set unless you cast it into a List or Tuple
- Hence Sets are unindexed and unordered.



The screenshot shows a code editor window titled 'main.py'. The code is as follows:

```
1
2
3 evenNumbers = {2, 4, 6, 8, 10, 12}
4 evenNumbers[0] = 5
5
6 print(evenNumbers)
7
8
```

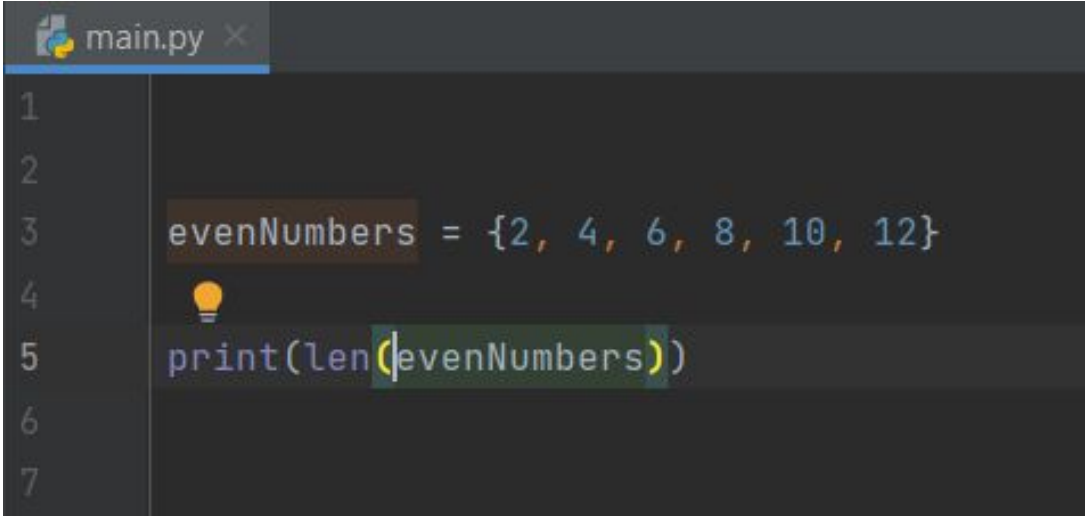
Line 3 defines a set named `evenNumbers` containing the values 2, 4, 6, 8, 10, and 12. Line 4 attempts to assign the value 5 to the first element of the set using indexing (`evenNumbers[0]`). This is an invalid operation because sets are unindexed. Line 6 prints the set.

Set LENGTH

- You can check the number of items in a set by using the len() function.

Syntax

len(set)

A screenshot of a Python IDE window titled 'main.py'. The code is as follows:

```
1  
2  
3 evenNumbers = {2, 4, 6, 8, 10, 12}  
4  
5 print(len(evenNumbers))  
6  
7
```

The code defines a set named 'evenNumbers' containing the values {2, 4, 6, 8, 10, 12} and then prints the length of this set using the len() function. A lightbulb icon is visible next to the print statement, indicating a tip or suggestion.

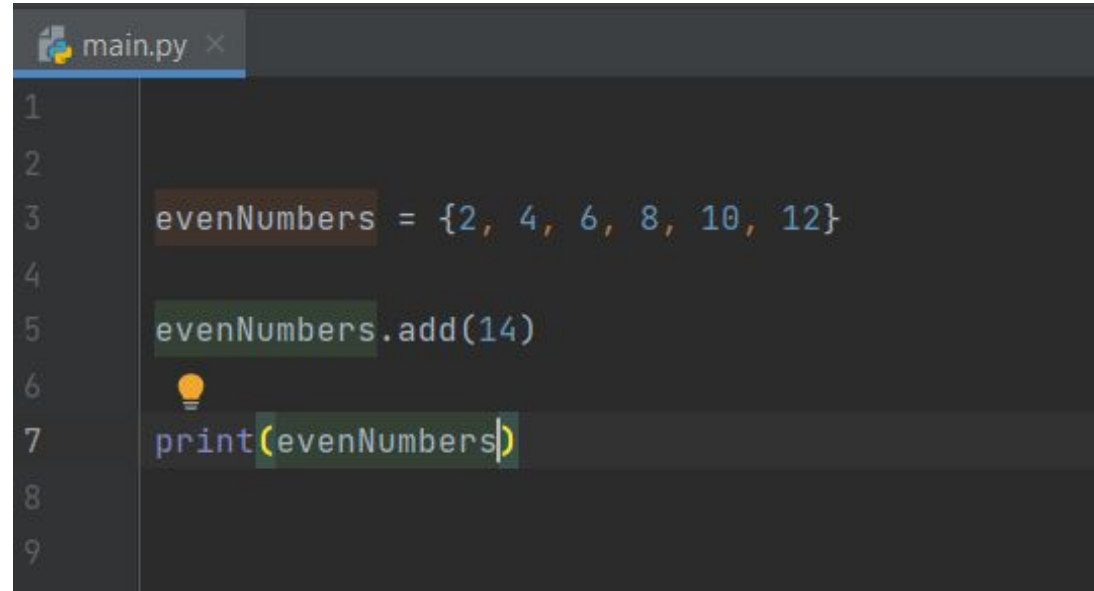
Set ADD ITEMS by ADD()

- `add()` adds an item at the END OF THE SET.

Syntax

`set.add(value)`

`numbers.add(7)`



```
main.py x
1
2
3 evenNumbers = {2, 4, 6, 8, 10, 12}
4
5 evenNumbers.add(14)
6
7 print(evenNumbers)
8
9
```

The screenshot shows a code editor window titled 'main.py'. It contains a Python script with the following lines: Line 3: `evenNumbers = {2, 4, 6, 8, 10, 12}`; Line 5: `evenNumbers.add(14)`; Line 7: `print(evenNumbers)`. A lightbulb icon is visible next to line 6, indicating a suggestion or tip. The code is syntax-highlighted, and the editor has a dark theme.

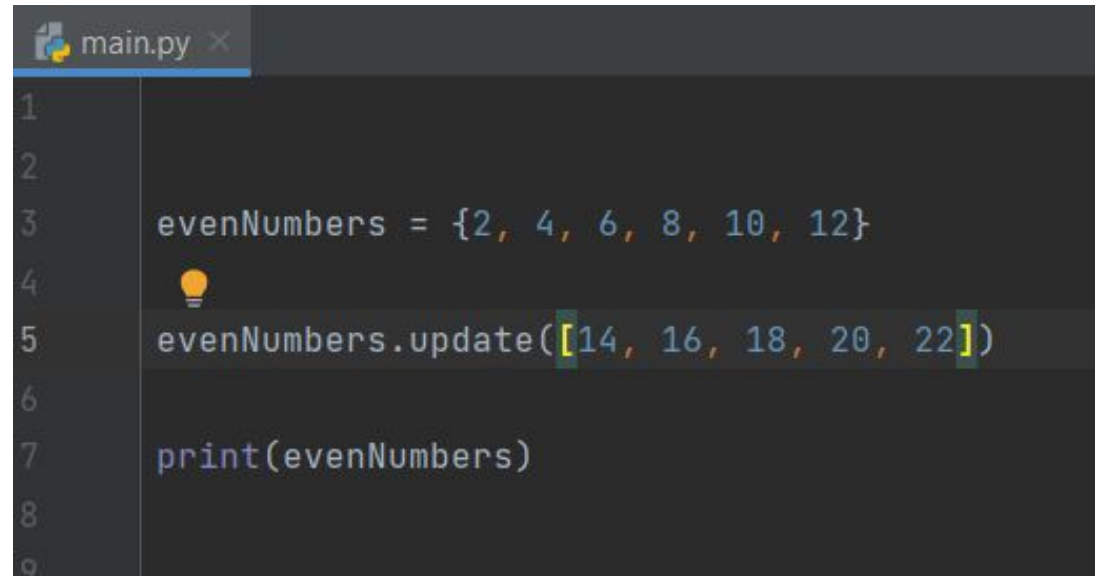
Set ADD ITEMS by UPDATE()

- `update()` allows multiple items to be added at the same time in the set.

Syntax

`set.update(list)`

`Number.update([8,9,10])`

A screenshot of a Python IDE window titled 'main.py'. The code is as follows:

```
1  
2  
3 evenNumbers = {2, 4, 6, 8, 10, 12}  
4  
5 evenNumbers.update([14, 16, 18, 20, 22])  
6  
7 print(evenNumbers)  
8  
9
```

The code demonstrates creating a set 'evenNumbers' with values {2, 4, 6, 8, 10, 12} and then updating it with a list [14, 16, 18, 20, 22]. A lightbulb icon is visible next to the update method call on line 5.

Set DELETING ITEMS by REMOVE()

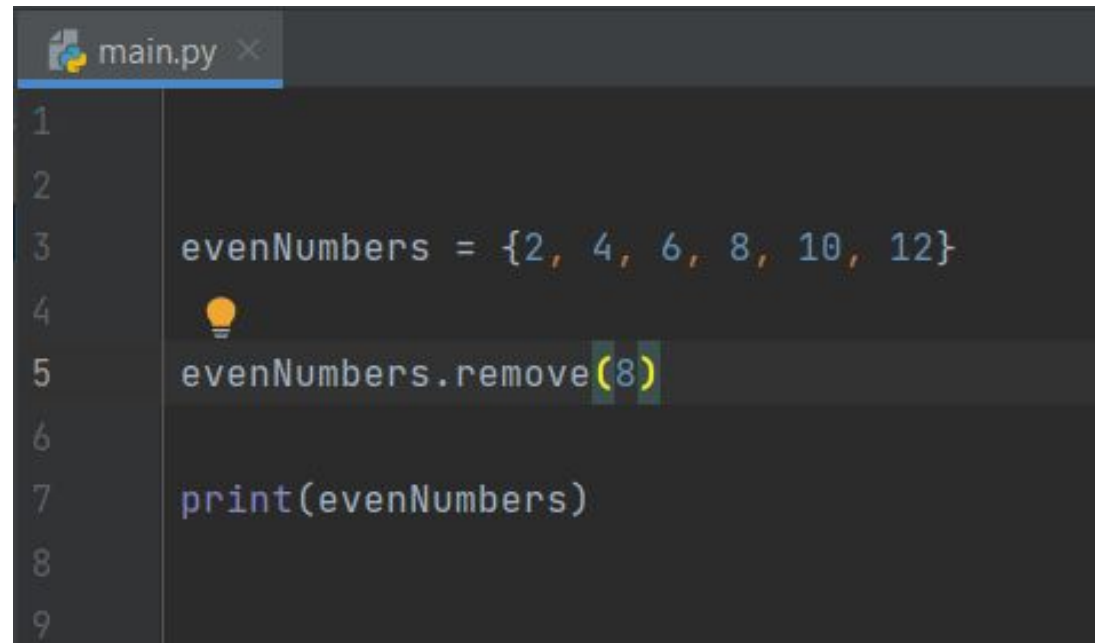
- `remove()` deletes an item based on their value.

Syntax

`set.remove(value)`

`numbers.remove(1)`

Note: if the value doesn't exist in the set it will be counted as an ERROR



The screenshot shows a code editor window titled 'main.py'. It contains the following Python code:

```
1
2
3  evenNumbers = {2, 4, 6, 8, 10, 12}
4
5  evenNumbers.remove(8)
6
7  print(evenNumbers)
8
9
```

A lightbulb icon is visible next to line 5, indicating a suggestion or tip. The code demonstrates creating a set of even numbers and then removing the value 8 from it.

Set DELETING ITEMS by DISCARD()

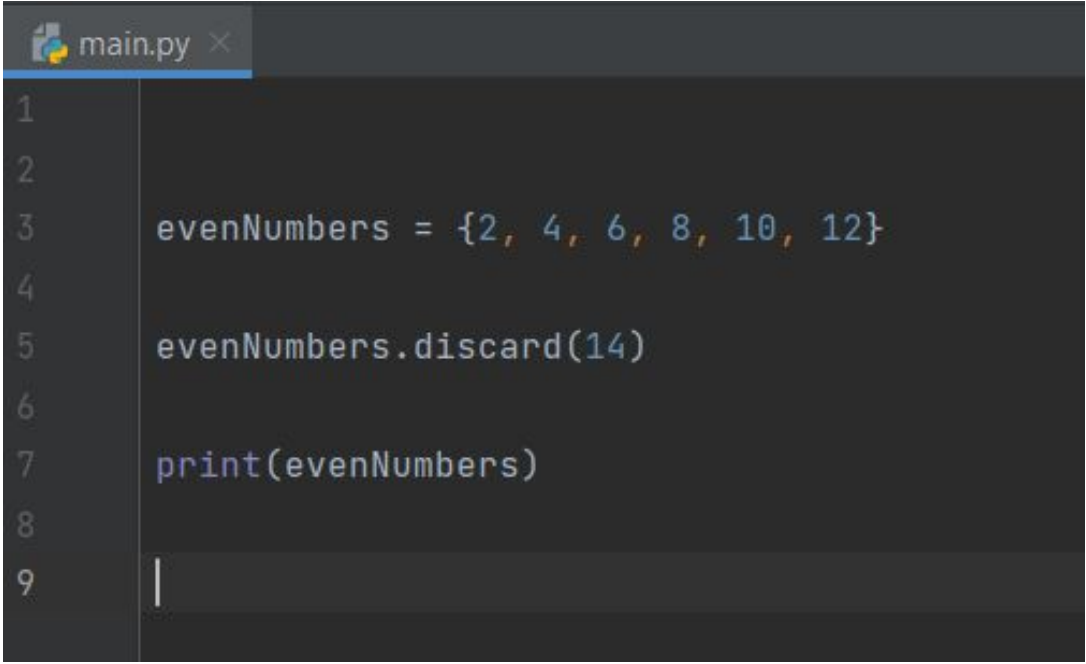
- `discard()` deletes an item based on their value.

Syntax

`set.discard(value)`

`numbers.discard(1)`

Note: if the value doesn't exist in the set it will NOT be counted as an ERROR



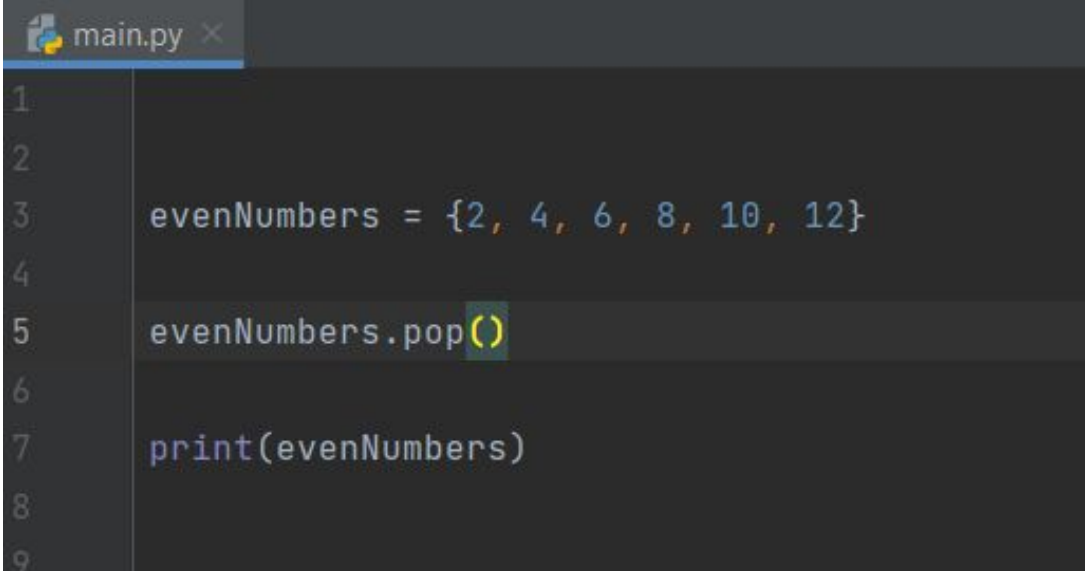
```
1  
2  
3 evenNumbers = {2, 4, 6, 8, 10, 12}  
4  
5 evenNumbers.discard(14)  
6  
7 print(evenNumbers)  
8  
9 |
```

Set DELETING ITEMS by POP()

- `pop()` deletes the first item in the Set.

Syntax

`set.pop()`



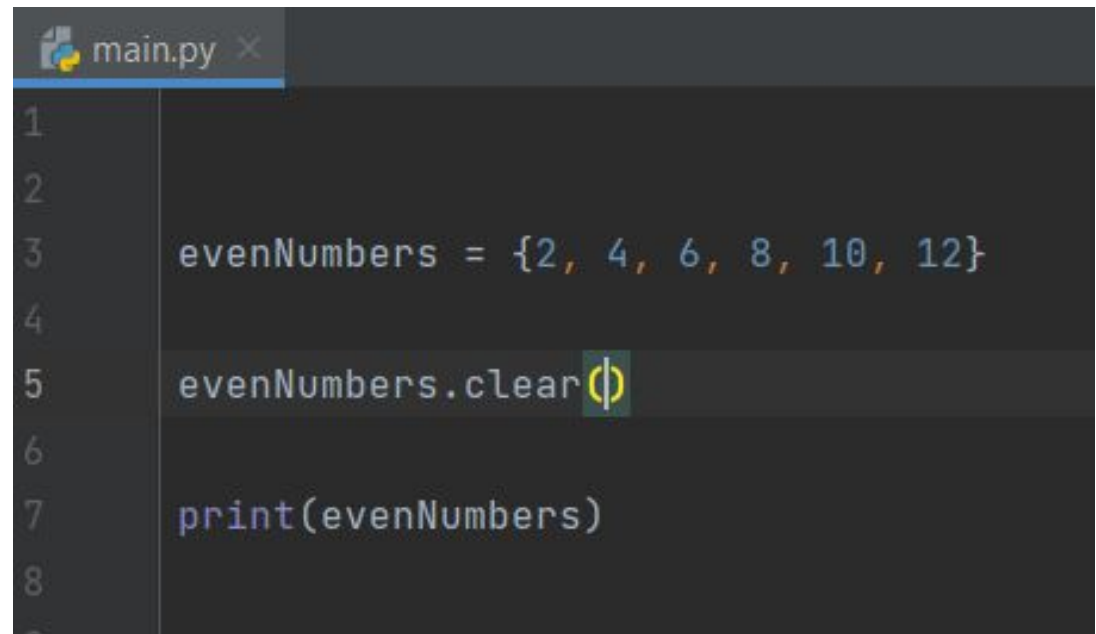
```
main.py ×  
1  
2  
3 evenNumbers = {2, 4, 6, 8, 10, 12}  
4  
5 evenNumbers.pop()  
6  
7 print(evenNumbers)  
8  
9
```

Clearing a Set

- `clear()` deletes all the value in a set.

Syntax

`set.clear()`

A screenshot of a Python IDE window titled 'main.py'. The code is as follows:

```
1  
2  
3 evenNumbers = {2, 4, 6, 8, 10, 12}  
4  
5 evenNumbers.clear()  
6  
7 print(evenNumbers)  
8
```

The code demonstrates creating a set of even numbers and then clearing it. The `clear()` method is called on the `evenNumbers` set.

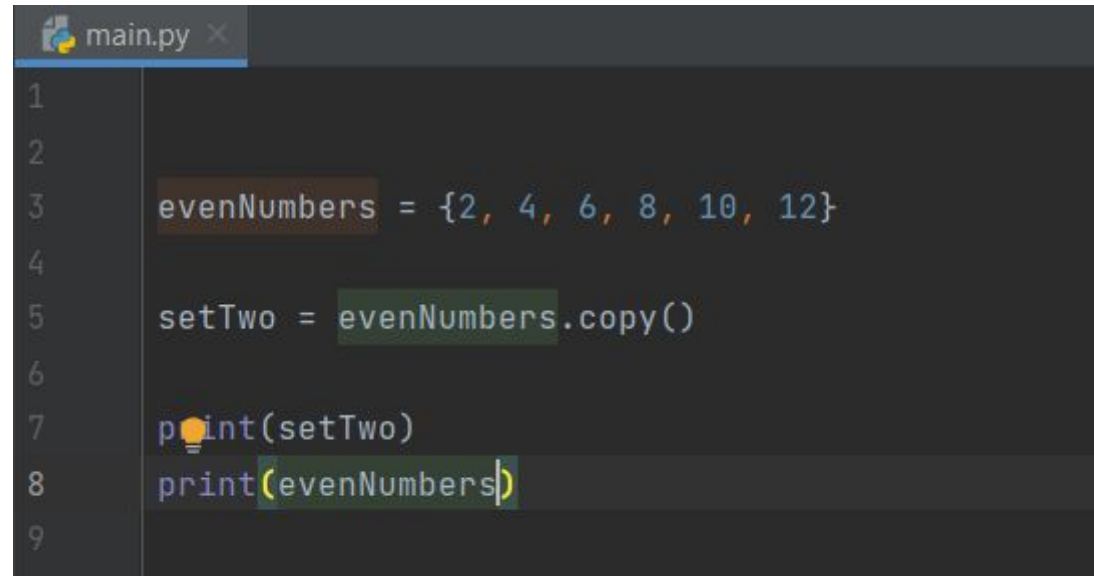
Copying a Set

- `copy()` copies the whole set which can be assigned to a new set.

Syntax

setOne = {1,2,3,4,5}

setTwo = *setOne*.copy()

A screenshot of a Python IDE window titled 'main.py'. The code is as follows:

```
1  
2  
3 evenNumbers = {2, 4, 6, 8, 10, 12}  
4  
5 setTwo = evenNumbers.copy()  
6  
7 print(setTwo)  
8 print(evenNumbers)  
9
```

UNION Set

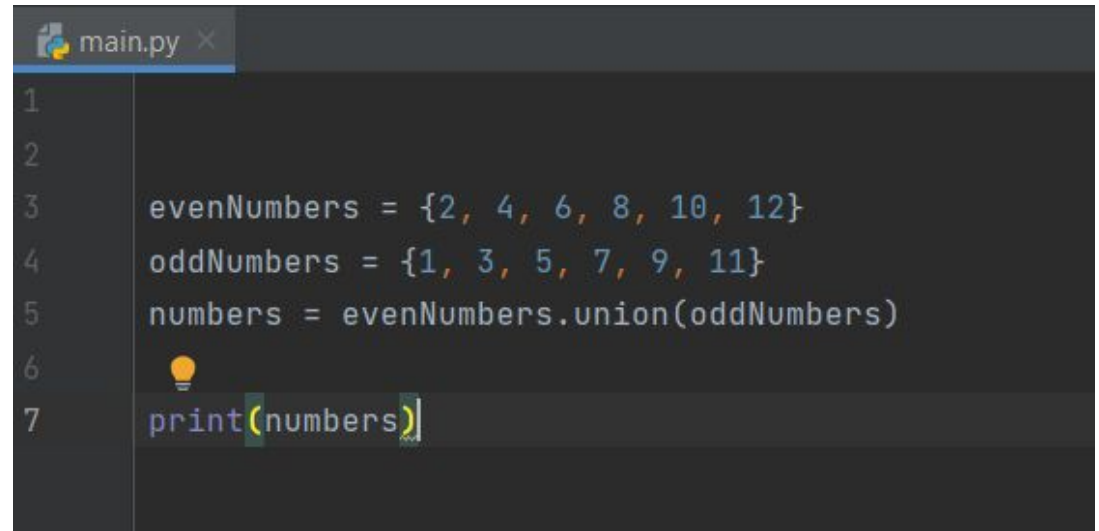
- `union()` returns a set containing all the values of the two sets.

Syntax

setOne = {1,2,3,4,5}

setTwo = {6,7,8,9,10}

setThree = setOne.union(setTwo)

A screenshot of a Python IDE window titled 'main.py'. The code is as follows:

```
1  
2  
3 evenNumbers = {2, 4, 6, 8, 10, 12}  
4 oddNumbers = {1, 3, 5, 7, 9, 11}  
5 numbers = evenNumbers.union(oddNumbers)  
6  
7 print(numbers)
```

A lightbulb icon is visible next to line 6, and the cursor is at the end of line 7.

DIFFERENCE Set

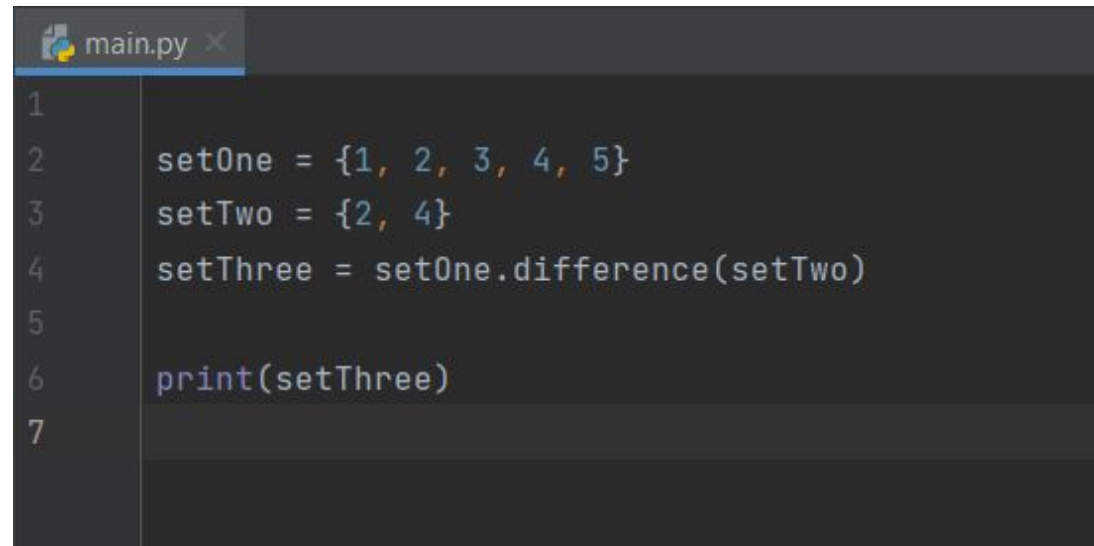
- `difference()` returns a set containing the values that only exist on the left set and not on the right set.

Syntax

setOne = {1,2,3,4,5}

setTwo = {3,4}

setThree = *setOne.difference(setTwo)*

A screenshot of a Python IDE window titled 'main.py'. The code is as follows:

```
1  
2  setOne = {1, 2, 3, 4, 5}  
3  setTwo = {2, 4}  
4  setThree = setOne.difference(setTwo)  
5  
6  print(setThree)  
7
```

INTERSECTION Set

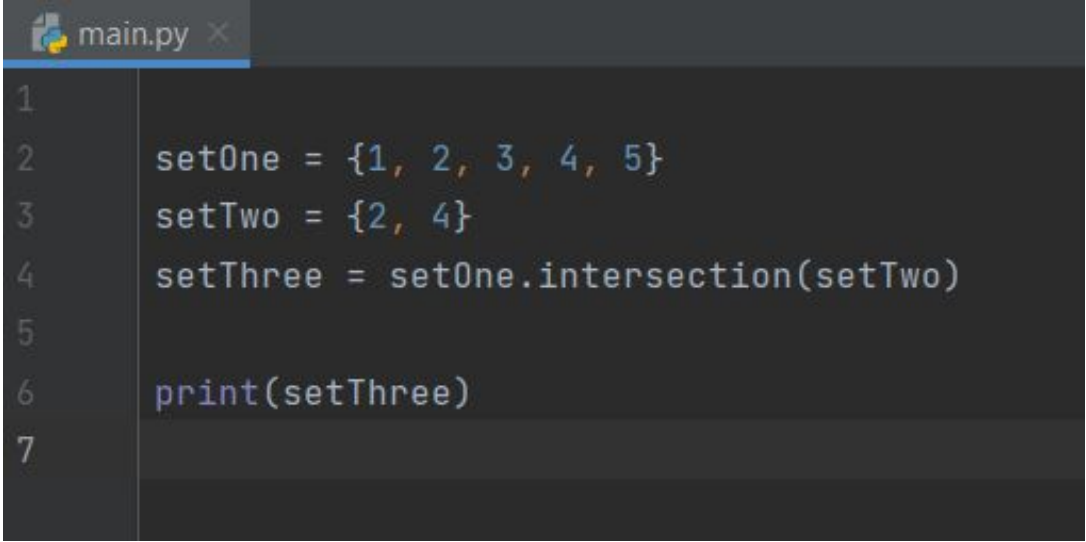
- `intersection()` returns a set containing the values that exist both on the two sets.

Syntax

setOne = {1,2,3,4,5}

setTwo = {3,4}

*setThree =
setOne.intersection(setTwo)*



```
main.py x
1
2 setOne = {1, 2, 3, 4, 5}
3 setTwo = {2, 4}
4 setThree = setOne.intersection(setTwo)
5
6 print(setThree)
7
```

SYMMETRIC DIFFERENCE Set

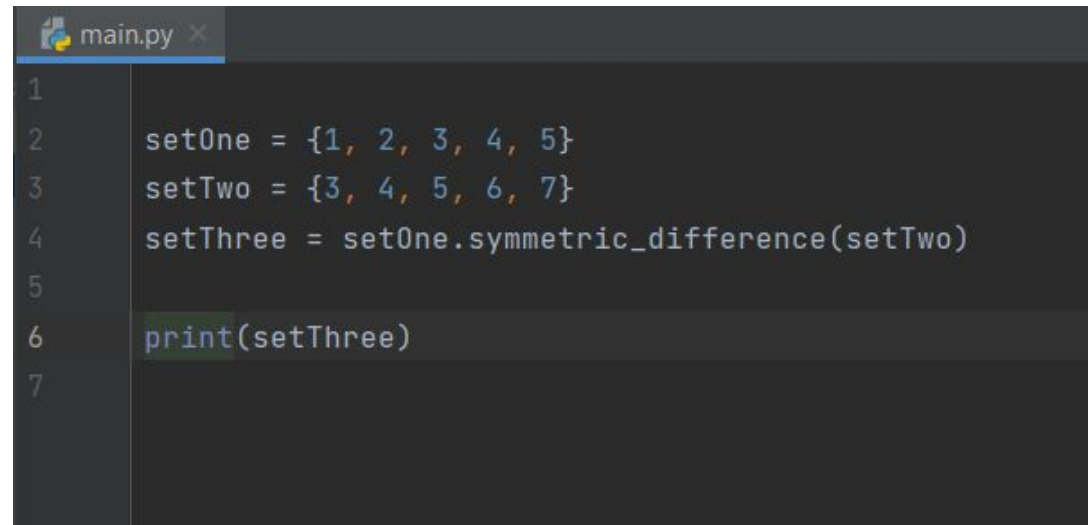
- `symmetric_difference()` returns a set containing all values that exists EXCLUSIVELY on each set.

Syntax

setOne = {1,2,3,4,5}

setTwo = {3,4,5,6,7}

setThree =
setOne.symmetric_difference(setTwo)

A screenshot of a Python code editor window titled 'main.py'. The code defines two sets, 'setOne' and 'setTwo', and then calculates their symmetric difference, storing the result in 'setThree'. Finally, it prints 'setThree'.

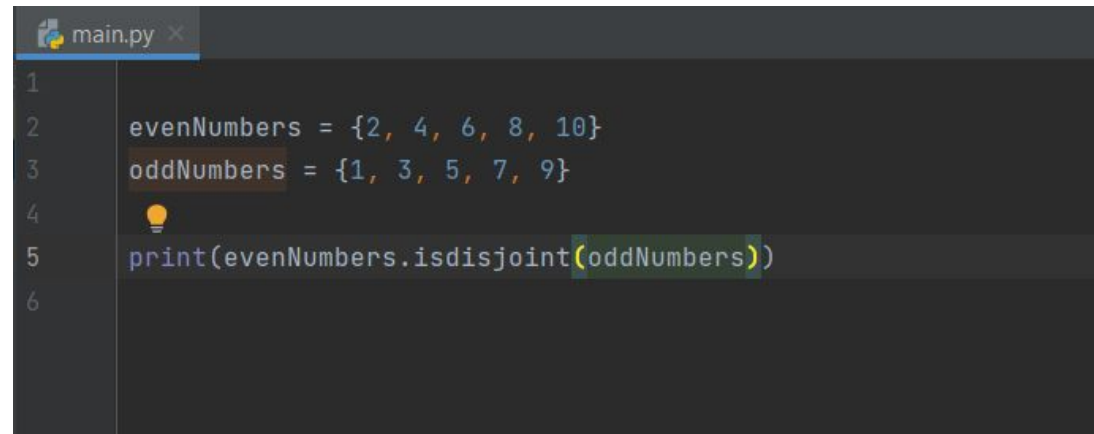
```
1  
2 setOne = {1, 2, 3, 4, 5}  
3 setTwo = {3, 4, 5, 6, 7}  
4 setThree = setOne.symmetric_difference(setTwo)  
5  
6 print(setThree)  
7
```

DISJOINT Set

- `isdisjoint()` returns a Boolean whether two sets have an intersection or not.

Syntax

`setOne.isdisjoint(setTwo)`



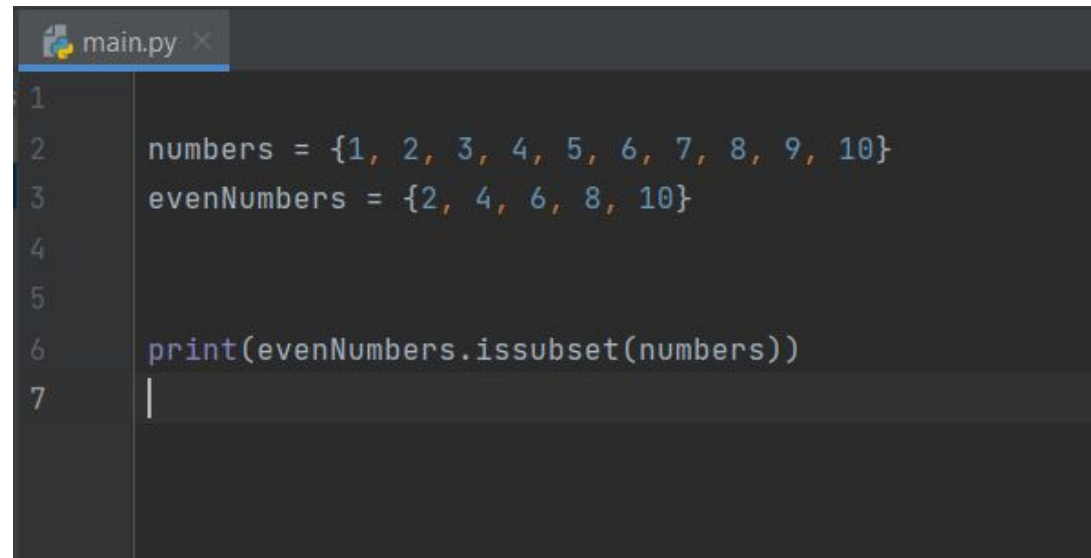
```
main.py x
1
2 evenNumbers = {2, 4, 6, 8, 10}
3 oddNumbers = {1, 3, 5, 7, 9}
4
5 print(evenNumbers.isdisjoint(oddNumbers))
6
```

SUBSET

- `issubset()` returns a Boolean whether the left set is contained in the right set.

Syntax

setOne.issubset(setTwo)



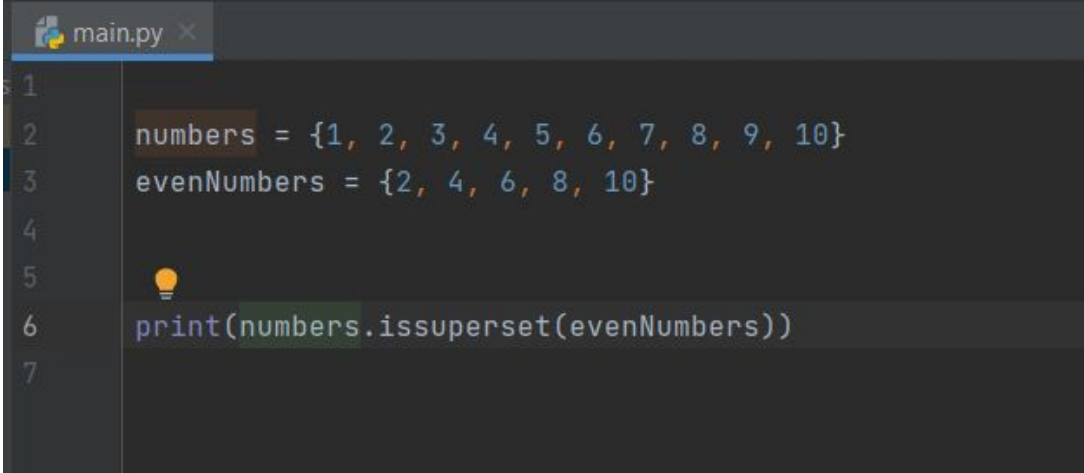
```
main.py x
1
2 numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
3 evenNumbers = {2, 4, 6, 8, 10}
4
5
6 print(evenNumbers.issubset(numbers))
7 |
```

SUPERSET

- `issuperset()` returns a Boolean whether the right set is contained in the left set.

Syntax

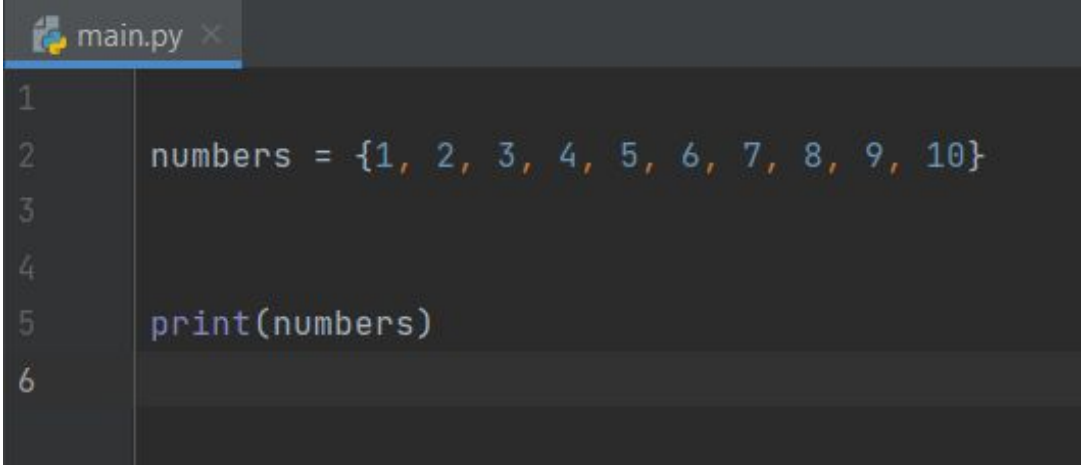
`setOne.issuperset(setTwo)`



```
main.py x
1
2 numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
3 evenNumbers = {2, 4, 6, 8, 10}
4
5
6 print(numbers.issuperset(evenNumbers))
7
```

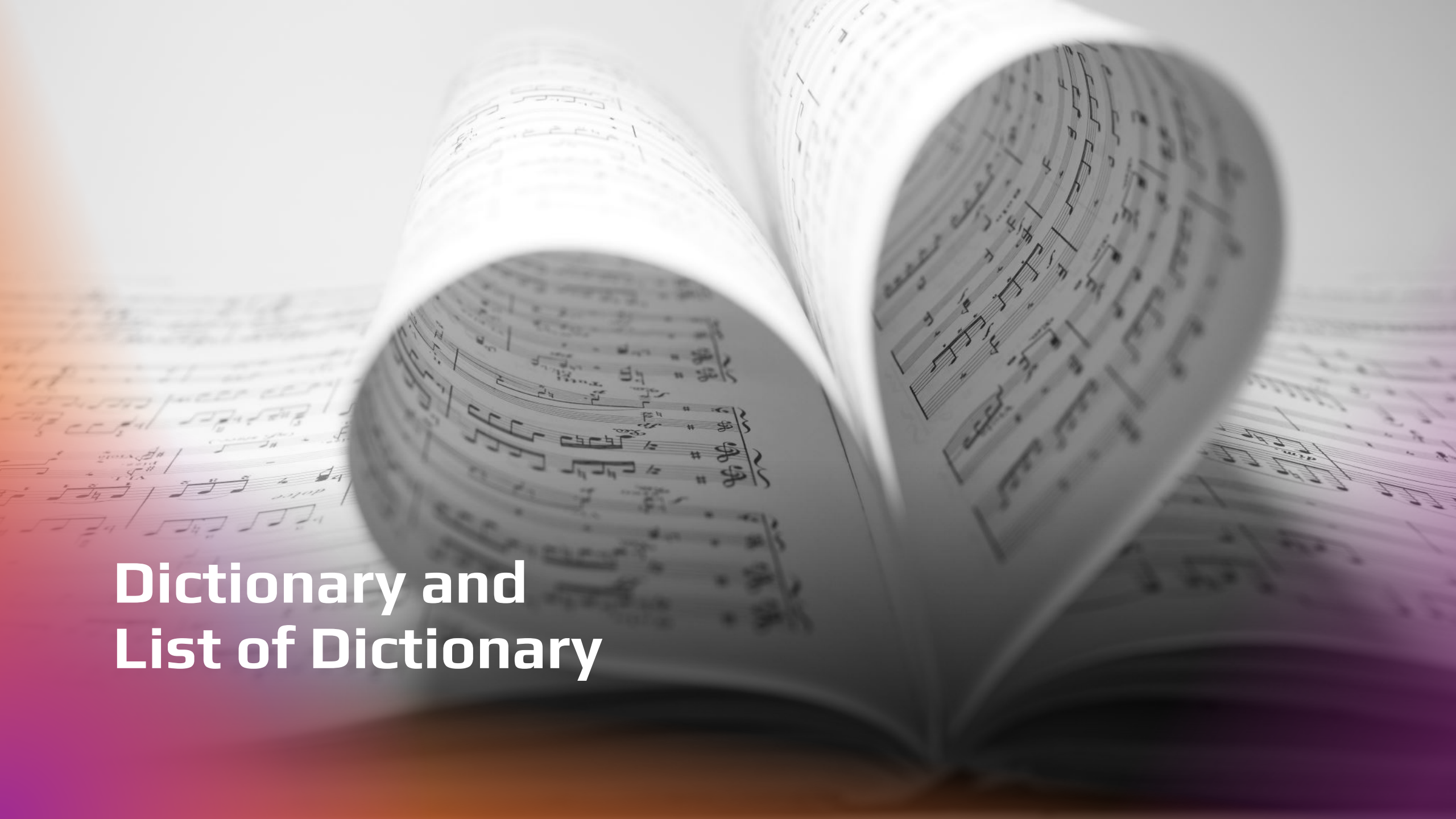
Casting SETS

- You can cast Sets to Tuples or List and VICEVERSA in the same way you cast other variables.



```
main.py x
1
2 numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
3
4
5 print(numbers)
6
```

The screenshot shows a code editor window titled 'main.py'. It contains a Python script with the following code: `numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}` on line 2 and `print(numbers)` on line 5. Line numbers 1 through 6 are visible on the left side of the editor.



Dictionary and List of Dictionary

What is Dictionary in Python?



Sets and dictionaries are ideal data structures to be used when your data has no intrinsic order, but does have a unique object that can be used to reference it (the reference object is normally a string, but can be any hashable type).



Python dictionaries are a fundamental data type for data storage and retrieval.



The dictionary is a built-in data structure that stores key: value pairs and can be accessed by either the key or the value.



Python dictionaries are unordered, and keys cannot be negative integers.

Dictionary

- A Collection of Key Pairs that is Unordered, Changeable and Indexed

Syntax

<i>Identifier = {</i>	<i>Student = {</i>
<i>key1: value1,</i>	<i>“name” : “James”,</i>
<i>key2: value2,</i>	<i>“course” : “BSIS”,</i>
<i>key3: value3,</i>	<i>“age”: 22</i>
<i>}</i>	<i>}</i>

Reading WHOLE Dictionary

- You can read a dictionary by printing the whole dictionary.

Syntax

`print(dictionary)`

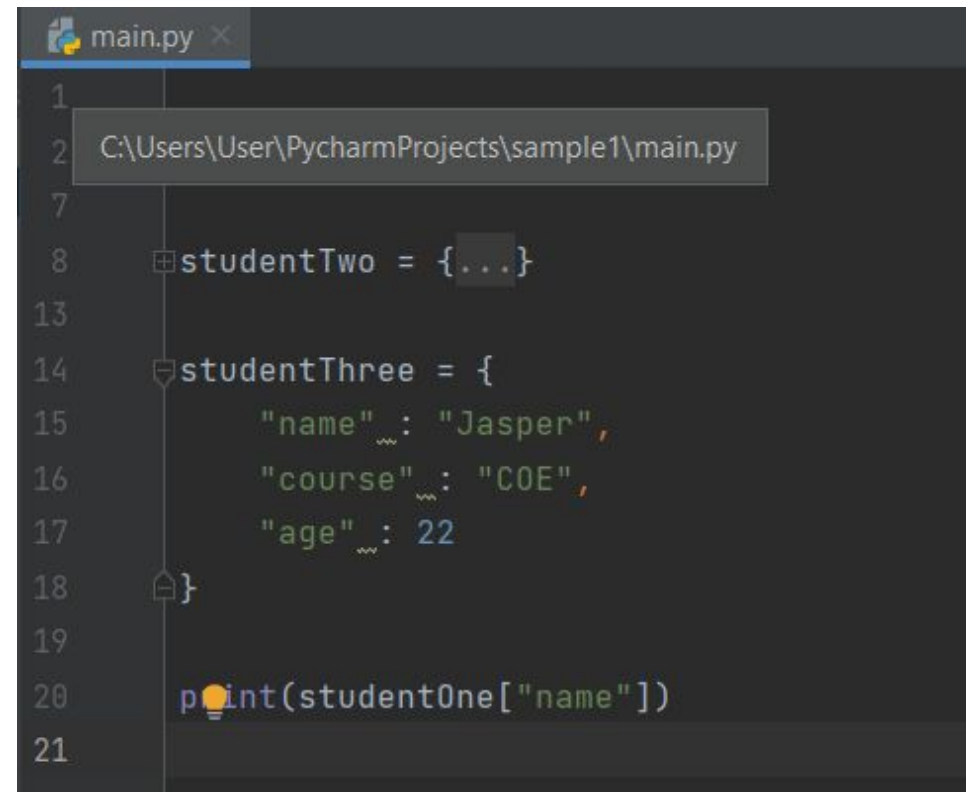
```
main.py x
2 studentOne = {
3     "name": "James",
4     "course": "BSIS",
5     "age": 21
6 }
7
8 studentTwo = {
9     "name": "John",
10    "course": "BSIT",
11    "age": 19
12 }
13
14 studentThree = {
15     "name": "Jasper",
16     "course": "COE",
17     "age": 22
18 }
19
20 print(studentOne)
21 print(studentTwo)
22 print(studentThree)
```

Reading Dictionary ITEMS

- You can read a dictionary by specifying the Key Value

Syntax

`print(dictionary[key])`



The screenshot shows a code editor window titled 'main.py' with a file path 'C:\Users\User\PycharmProjects\sample1\main.py'. The code defines two dictionaries: 'studentTwo' (line 8) and 'studentThree' (lines 14-18). 'studentThree' contains keys 'name', 'course', and 'age' with values 'Jasper', 'COE', and 22 respectively. Line 20 shows the command 'print(studentOne["name"])' with a yellow lightbulb icon indicating a suggestion or error.

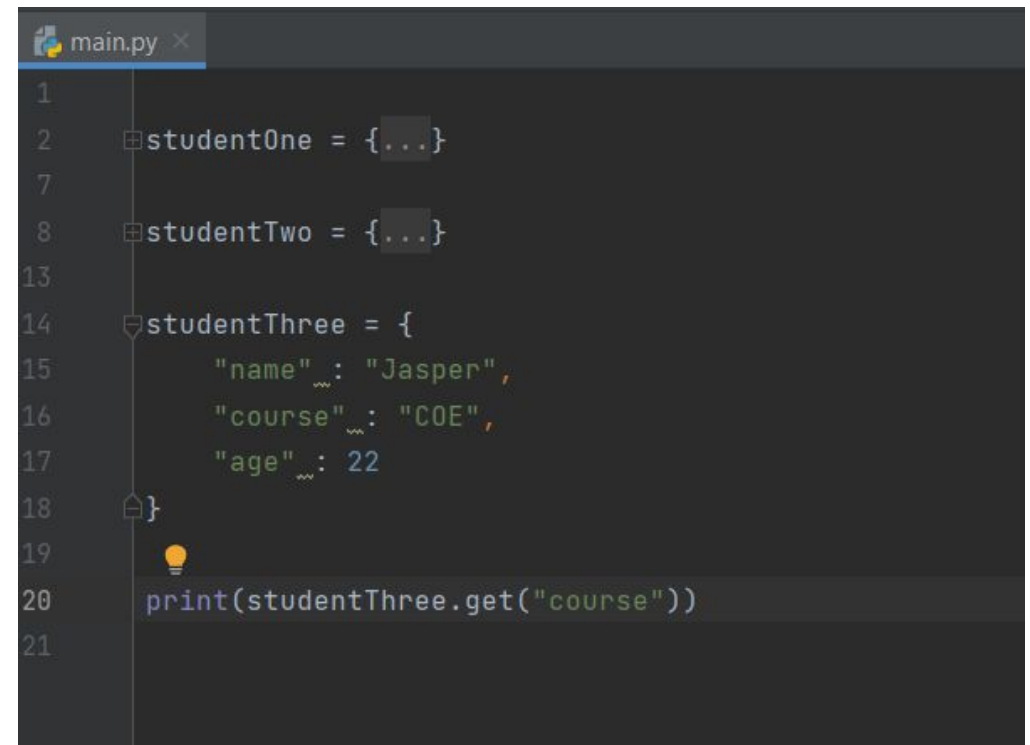
```
1
2 C:\Users\User\PycharmProjects\sample1\main.py
7
8 studentTwo = {...}
13
14 studentThree = {
15     "name": "Jasper",
16     "course": "COE",
17     "age": 22
18 }
19
20 print(studentOne["name"])
21
```

Reading Dictionary ITEMS GET

- You can also use GET to read a dictionary by specifying the Key Value

Syntax

`print(dictionary.get(key))`



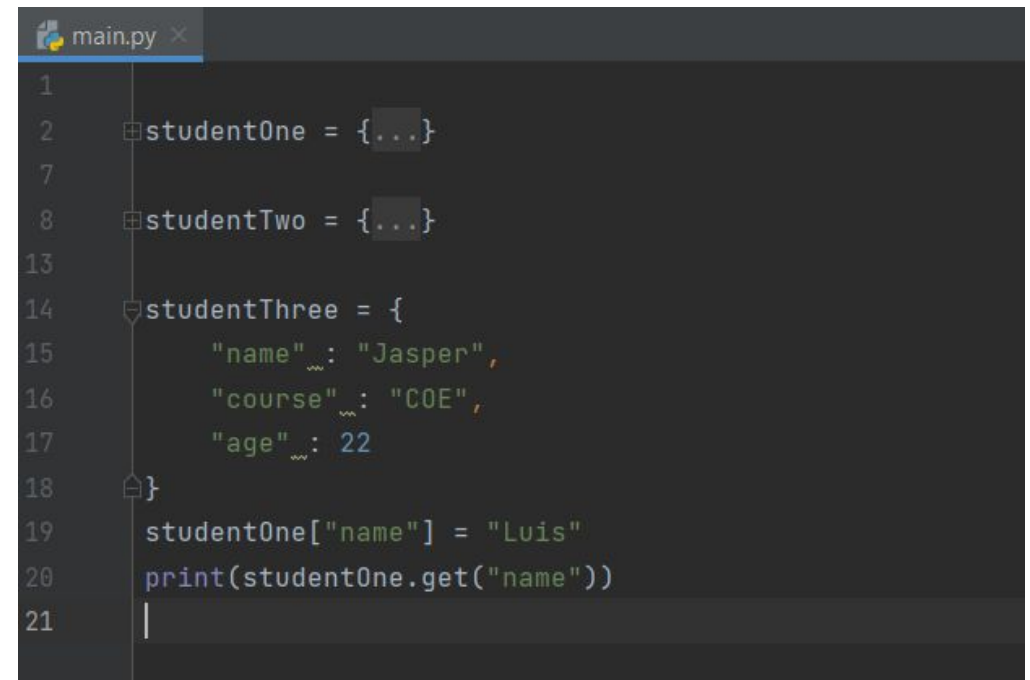
```
main.py x
1
2 studentOne = {...}
7
8 studentTwo = {...}
13
14 studentThree = {
15     "name": "Jasper",
16     "course": "COE",
17     "age": 22
18 }
19
20 print(studentThree.get("course"))
21
```

Assigning Dictionary ITEMS

- You can change a value of a certain item in a dictionary by specifying the Key Value and using the Assignment Operator “=”

Syntax

dictionary[key] = value



```
1
2  studentOne = {...}
7
8  studentTwo = {...}
13
14  studentThree = {
15      "name": "Jasper",
16      "course": "COE",
17      "age": 22
18  }
19  studentOne["name"] = "Luis"
20  print(studentOne.get("name"))
21
```

Dictionary LENGHT

- You can check the number of items (key pairs) in a Dictionary by using the len() function

Syntax

len(dictionary)

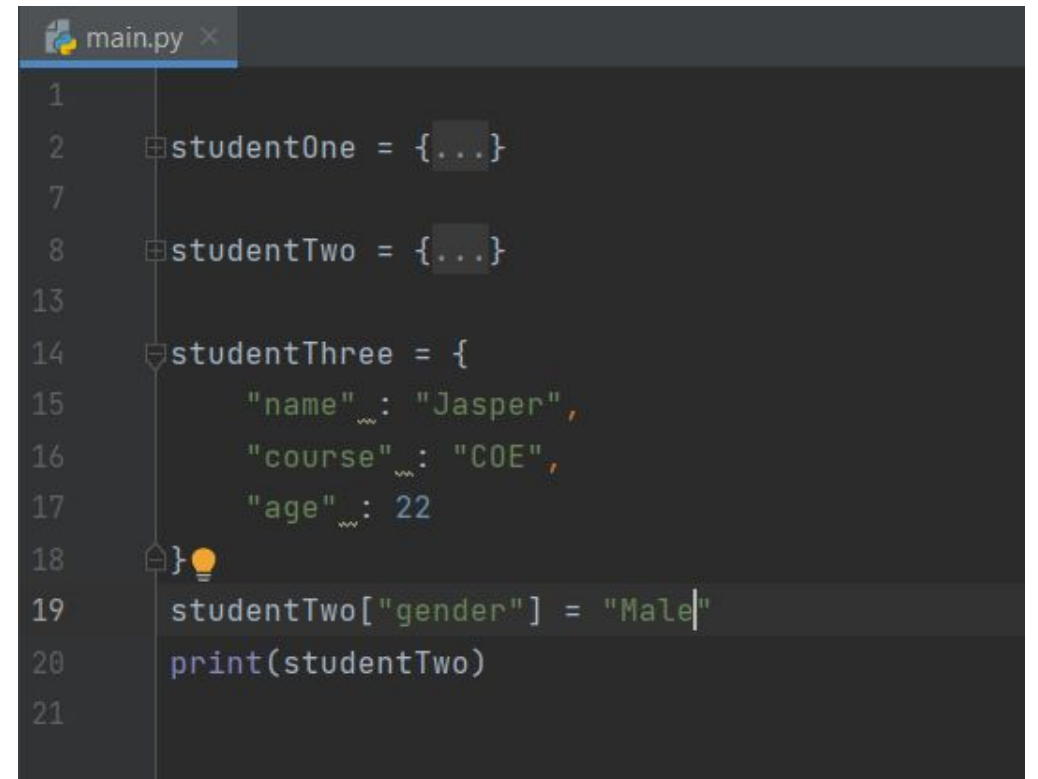
```
main.py x
1
2 studentOne = {...}
7
8 studentTwo = {...}
13
14 studentThree = {
15     "name": "Jasper",
16     "course": "COE",
17     "age": 22
18 }
19
20 print(len(studentTwo))
21 |
```

Dictionary ADD ITEMS

- You can add items on a dictionary just by assigning a non-existent key value in the dictionary

Syntax

dictionary[key] = value

A screenshot of a Python code editor window titled 'main.py'. The code defines three dictionaries: 'studentOne' and 'studentTwo' are empty dictionaries, and 'studentThree' is a dictionary with keys 'name', 'course', and 'age'. On line 19, the code adds a new key-value pair to 'studentTwo' by assigning 'studentTwo["gender"] = "Male"'. Line 20 contains 'print(studentTwo)'.

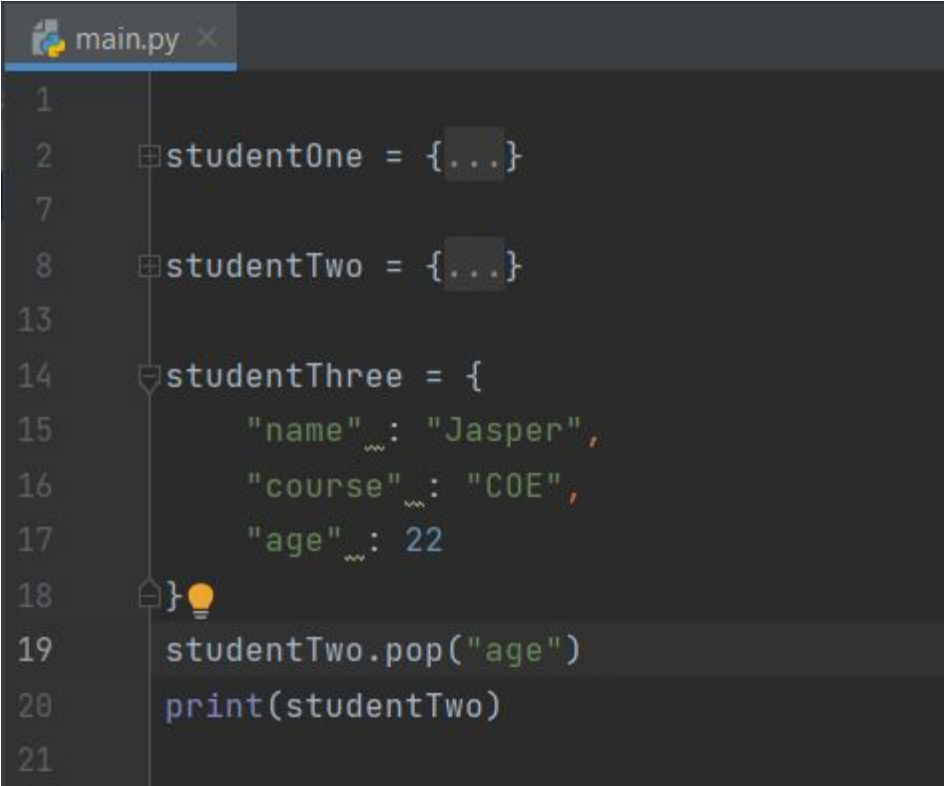
```
1
2 studentOne = {...}
7
8 studentTwo = {...}
13
14 studentThree = {
15     "name": "Jasper",
16     "course": "COE",
17     "age": 22
18 }
19 studentTwo["gender"] = "Male"
20 print(studentTwo)
21
```


Dictionary DELETING ITEMS by POP()

- pop() deletes an item based on their key value.

Syntax

dictionary.pop(key)



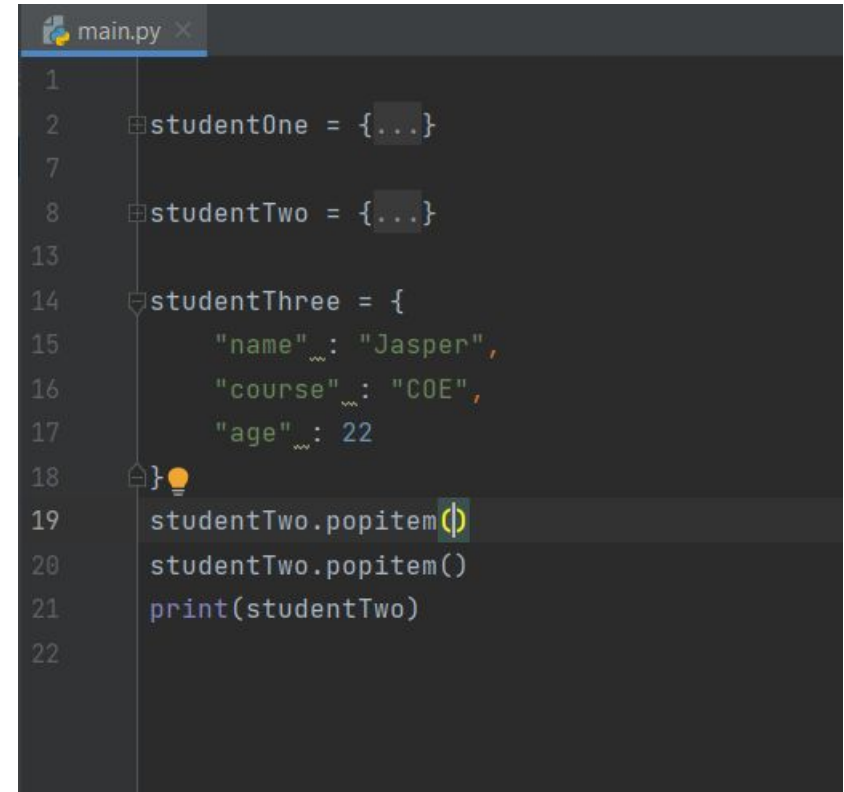
```
main.py x
1
2 studentOne = {...}
7
8 studentTwo = {...}
13
14 studentThree = {
15     "name": "Jasper",
16     "course": "COE",
17     "age": 22
18 }
19 studentTwo.pop("age")
20 print(studentTwo)
21
```

Dictionary DELETING ITEMS by POPITEM()

- `popitem()` deletes the last inserted item on the dictionary.

Syntax

`dictionary.popitem()`



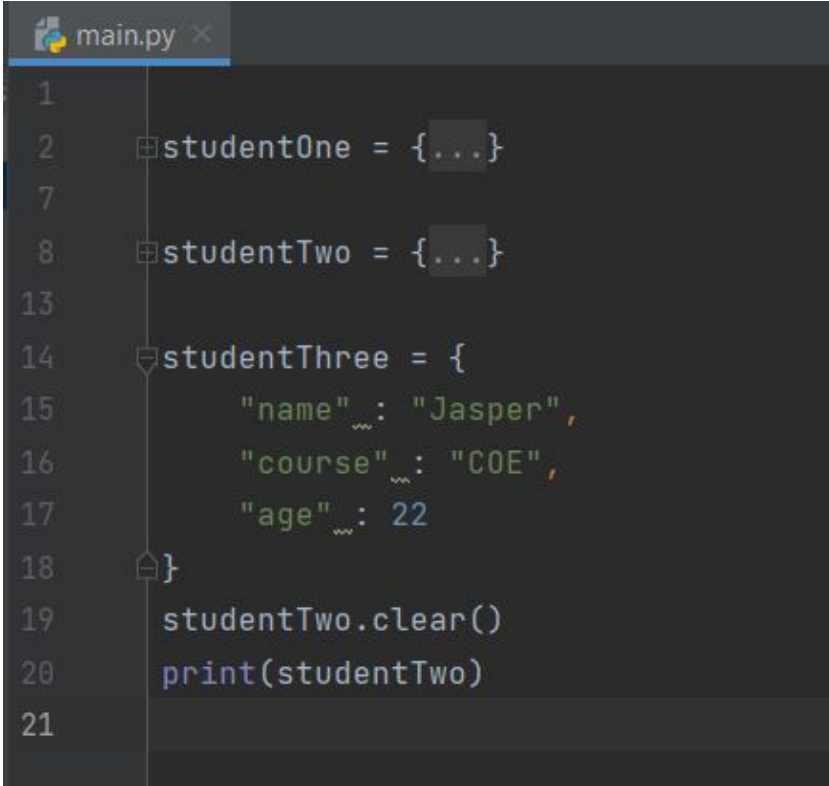
```
main.py x
1
2 studentOne = {...}
7
8 studentTwo = {...}
13
14 studentThree = {
15     "name": "Jasper",
16     "course": "COE",
17     "age": 22
18 }
19 studentTwo.popitem()
20 studentTwo.popitem()
21 print(studentTwo)
22
```

Clearing a Dictionary

- `clear()` deletes all the items in a Dictionary.

Syntax

`dictionary.clear()`



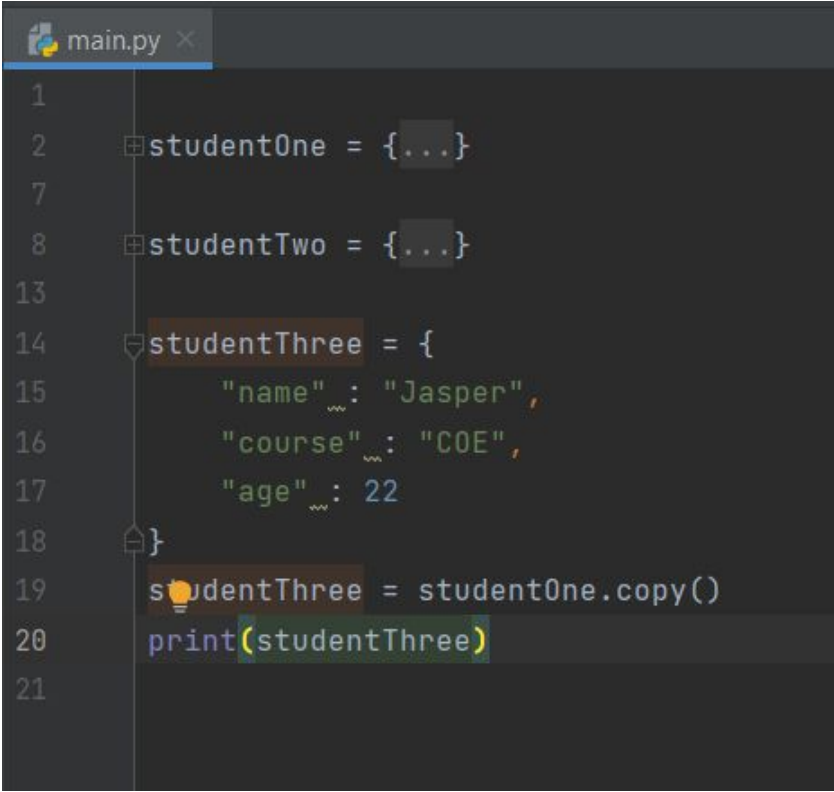
```
main.py x
1
2 studentOne = {...}
7
8 studentTwo = {...}
13
14 studentThree = {
15     "name": "Jasper",
16     "course": "COE",
17     "age": 22
18 }
19 studentTwo.clear()
20 print(studentTwo)
21
```

Copying a Dictionary

- `copy()` copies the whole dictionary which can be assigned to a new dictionary.

Syntax

```
student =  
{"name": "James", "gender": "male"}  
studentTwo = student.copy()
```



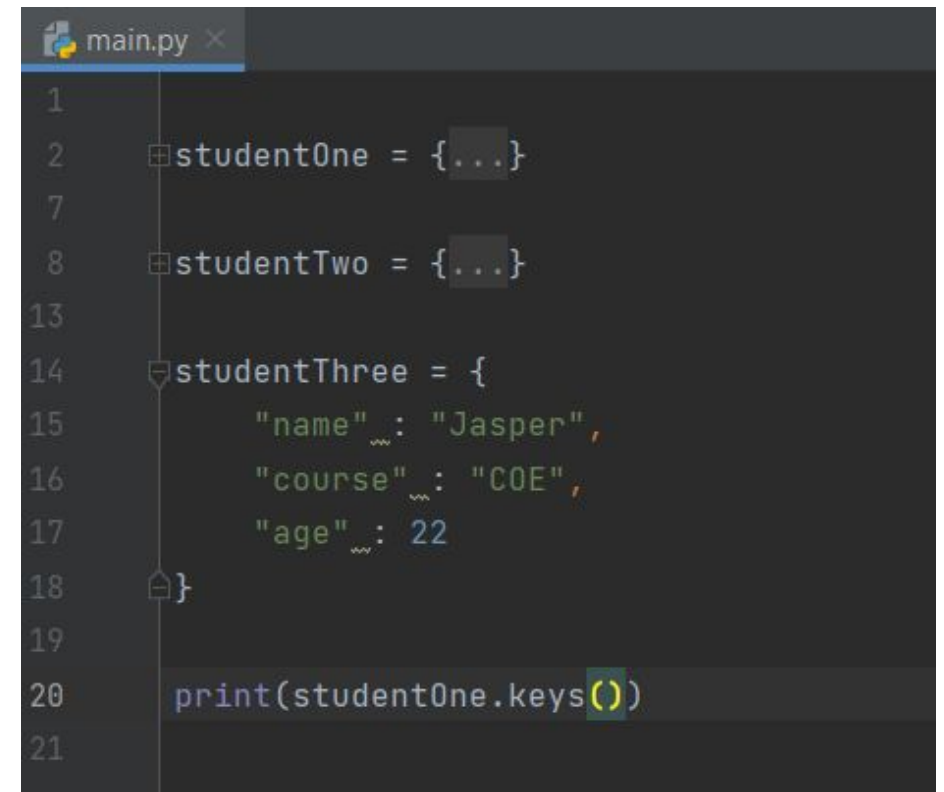
```
main.py x  
1  
2 studentOne = {...}  
7  
8 studentTwo = {...}  
13  
14 studentThree = {  
15     "name": "Jasper",  
16     "course": "COE",  
17     "age": 22  
18 }  
19 studentThree = studentOne.copy()  
20 print(studentThree)  
21
```

Getting ALL Keys in Dictionary

- `keys()` returns a list that contains all the keys inside your dictionary.

Syntax

`dictionary.keys()`



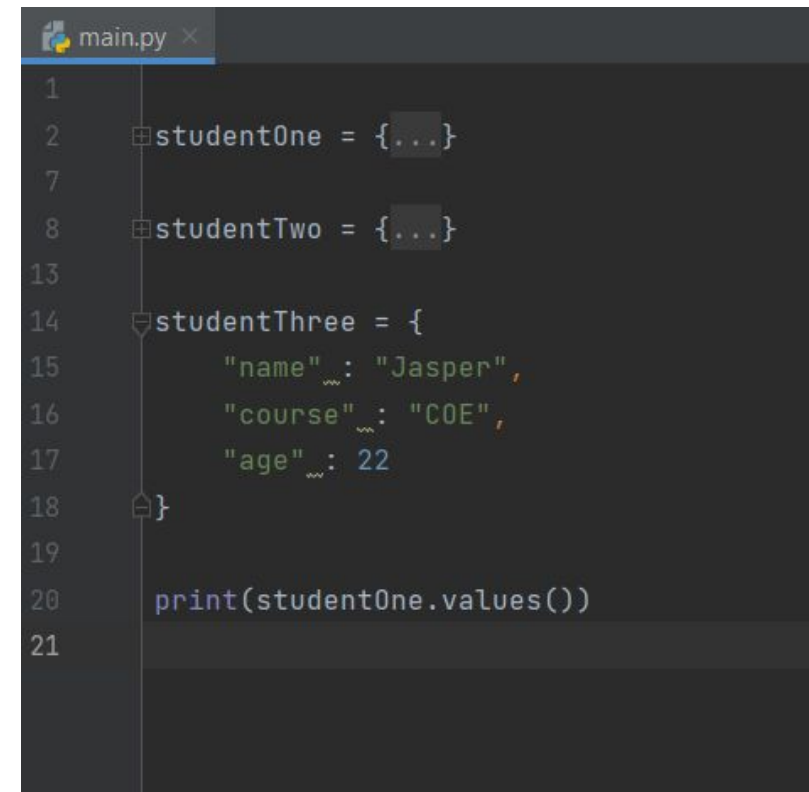
```
main.py x
1
2 studentOne = {...}
7
8 studentTwo = {...}
13
14 studentThree = {
15     "name": "Jasper",
16     "course": "COE",
17     "age": 22
18 }
19
20 print(studentOne.keys())
21
```

Getting ALL Values in Dictionary

- `values()` returns a list that contains all the value inside your dictionary.

Syntax

`dictionary.values()`



```
main.py x
1
2 studentOne = {...}
7
8 studentTwo = {...}
13
14 studentThree = {
15     "name": "Jasper",
16     "course": "COE",
17     "age": 22
18 }
19
20 print(studentOne.values())
21
```

List of Dictionaries

- Dictionaries inside a list
- A dictionary of lists is a data structure where:
- Each key in the dictionary represents a field or attribute (like description, price, qty, etc.).
- The value associated with each key is a list of values for that attribute.

```
main.py
1
2 studentOne = {...}
7
8 studentTwo = {...}
13
14 studentThree = {
15     "name": "Jasper",
16     "course": "COE",
17     "age": 22
18 }
19 students = [studentOne, studentTwo, studentThree]
20 print(students)
21
```

```
main.py x
1
2 studentOneAttributes = {
3     "height": 190,
4     "weight": 70,
5     "skin": "fair"
6 }
7 studentOne = {
8     "name": "James",
9     "course": "BSIS",
10    "age": 21,
11    "physical": studentOneAttributes
12 }
13
14 print(studentOne.get("physical"))
```

Nested Dictionaries

- A Dictionaries inside a dictionary
- As described before, Python dictionaries are a type of data structure that allows for storing data in key-value pairs.
- Nested dictionaries are dictionaries that are stored as values within another dictionary.

